



Myelith

Ein dezentrales Netzwerk, in dem Konsensarbeit
ein agentisches Sprachmodell betreibt

Technisches Whitepaper · Entwurf v0.3

Joschka Benjamin Hänsler

August 2026

Lizenz: CC BY-SA 4.0 · Kein Token existiert · Kommentare willkommen

Bis Version 0.1 unter dem Namen Myelin geführt; alle Fassungen sind über die Konzept-DOI verbunden.

Inhalt

Abstract	3
1. Einleitung	3
2. Verwandte Arbeiten und Abgrenzung	4
3. Architektur	6
4. Compute Layer: Modell-Sharding und Pods	8
5. Tokenomics — Der Burn-and-Mint-Kreislauf	9
6. Verifikation — Formales Modell	11
7. Training und Modellentwicklung	15
8. Agent Layer	17
9. Vertraulichkeit und Risikomodell	19
10. Governance und Modell-Herkunft	19
11. Offene Forschungsfragen	20
Anhang A: Kern-Datentypen und Referenzalgorithmen	21
Anhang B: Anreiz-Herleitung	22
Literatur	27

Abstract

Proof-of-Work-Blockchains erkaufen ihre Sicherheit durch den Einsatz von Rechenleistung und Energie, die dabei geleistete Arbeit selbst verwerfen sie jedoch vollständig. Dezentrale KI-Netzwerke stellen nützliche Rechenleistung bereit, sichern damit aber keinen Ledger. Myelith vereint beide Funktionen: Miner betreiben gemeinsam ein großes agentisches Sprachmodell (das *Netzwerkmodell*) über Pipeline-Parallelismus, und dieselbe, kryptografisch nachgewiesene Inferenzarbeit ("Proof of Inference", PoI) bestimmt die Vergütung und speist das Stimmgewicht des Konsenses. Der native Coin MYL schließt den Wertkreislauf: Nutzer verbrennen MYL gegen Inferenz-Credits, Miner erhalten neu geprägte MYL proportional zur verifizierten Arbeit (Burn-and-Mint-Gleichgewicht). Wir spezifizieren (i) eine Schichtenarchitektur, die Konsens-Latenz von Inferenz-Latenz entkoppelt, (ii) ein Verifikationsmodell auf Basis vollständig ganzzahliger Ausführung: Da die Ganzzahladdition assoziativ ist, entsteht Bitgleichheit ohne jede Vorschrift über die Reihenfolge der Operationen, sodass heterogene Hardware ohne Durchsatzverlust und ohne Wettbewerbsnachteil teilnimmt; ergänzt um eingeschleuste Kontrollsegmente gegen den einmaligen gezielten Eingriff und einen wählbaren Modus bestätigter Auslieferung; wir dokumentieren zudem, warum die gleitkommabasierten Gegenentwürfe nicht tragen, (iii) eine Tokenomik mit quantifizierbarer Sicherheitsbedingung ($S_{\min} = g/p^2$), (iv) ein Trainingsverfahren, das die freie Kapazität nutzt, Datenherkunft statt Dateninhalt prüft und dem Netzwerk erlaubt, sein Modell schrittweise wachsen zu lassen, (v) einen Agent Layer, der die Grenze der Verifizierbarkeit an der Schnittstelle zur Außenwelt sichtbar macht und Schäden über konsensdurchgesetzte Session-Grenzen begrenzt, (vi) ein explizites Vertraulichkeits-Risikomodell mit Nutzungsklassen und (vii) Kern-Datentypen und Referenzalgorithmen einer quelloffenen Implementierung. Offene Punkte, die Ausgabequalität ganzzahlig quantisierter Modelle in der Zielgrößenordnung, die Vollständigkeit der Ausführungsspezifikation, die Ununterscheidbarkeit der Kontrollsegmente und der 50-%-Redundanz-Overhead, werden als Messfragen mit zugeordneten Meilensteinen benannt. Version 0.3 ergänzt die Messung an einer Referenzimplementierung: Die vollständig ganzzahlige Ausführung eines Sprachmodells ist bitidentisch reproduzierbar und kostet gegenüber der Gleitkomma-Referenz 4,3 Prozent Perplexität; die Kosten entstehen dabei nicht in der Gewichtsdarstellung, sondern in den Zwischenstufen des Rechenpfades.

1. Einleitung

1.1 Zwei verschwendete Ressourcen

Das Bitcoin-Netzwerk verbraucht Energie im Umfang eines mittleren Industriestaats, um SHA-256-Hashes zu berechnen, deren einziger Zweck der Nachweis ihres eigenen Verbrauchs ist. Gleichzeitig konzentriert sich die Rechenleistung für große Sprachmodelle in den Rechenzentren weniger Konzerne; Zugang, Preisgestaltung und Modellverhalten unterliegen zentraler Kontrolle. Beide Systeme verschwenden, das eine Rechenarbeit, das andere die Möglichkeit offener Teilhabe.

Die naheliegende Synthese (Mining-Arbeit ist KI-Arbeit) scheitert bislang an drei technischen Hürden:

1. **Verifikation:** Klassisches PoW ist asymmetrisch (schwer zu lösen, trivial zu prüfen). LLM-Inferenz ist symmetrisch: Die Prüfung kostet annähernd so viel wie die Berechnung.
2. **Latenz:** Ein Modell, das nicht auf eine einzelne Maschine passt, muss über das Netz verteilt werden; naive Verteilung scheitert an WAN-Latenzen.
3. **Konsens-Kopplung:** Es ist unklar, wie aus "Ich habe Layer 40–60 korrekt berechnet" ein Blockrecht wird, ohne Grinding- und Kollusionsangriffe zu öffnen.

1.2 Beitrag

Dieses Papier schlägt eine Architektur vor, die diese drei Hürden nicht wegdefiniert, sondern mit jeweils expliziten Kosten adressiert: Verifikation über Redundanz plus Stichproben (Kosten: 50 % Overhead + Streitfristen), Latenz über Pipeline-Parallelismus mit latenzbewusster Pod-Bildung (Kosten: Sekundenbereich pro Pipeline-Durchlauf, kompensiert durch Micro-Batching und spekulatives Decoding), Konsens-Kopplung über die Trennung von schneller BFT-Blockproduktion und epochenweiser PoI-Abrechnung (Kosten: Vergütungsverzögerung um eine Streitfrist).

Gegenüber v0.1 ist die Verifikation grundlegend neu begründet. v0.1 verlangte bitidentische Gleitkomma-Ausführung und musste dafür die Reihenfolge aller Operationen vorschreiben, was Durchsatz kostet und einheitliches Rundungsverhalten der Hardware voraussetzt, eine Bedingung, die für die schnellen Rechenpfade moderner Beschleuniger nicht erfüllt ist. v0.2 führte die Inferenz stattdessen vollständig ganzzahlig aus. Da die Ganzzahladdition assoziativ ist, entsteht Bitgleichheit dann von selbst, unabhängig davon, wie die Hardware parallelisiert. Kapitel 6.3 dokumentiert, warum die gleitkommabasierten Alternativen, feste Reihenfolge, Toleranzvergleich und Software-Emulation, bei näherer Prüfung nicht tragen. Gegenüber v0.2 kommt mit Kapitel 6.9 ein Abschnitt hinzu, der erste Befunde der Referenzimplementierung aufnimmt: Die Bitbreite ist je Tensor festzulegen, und die Ausführungsspezifikation ist aus den Gewichten zu

gewinnen, nicht aus der Architekturbeschreibung.

Der Anspruch ist bewusst bescheidener als "besser als zentrale Anbieter": Der Redundanz-Overhead verbietet Effizienzführerschaft; Myelith konkurriert daher über Zensurrestistenz, Verfügbarkeit, offene Preisbildung und die Verwertung andernfalls brachliegender Hardware.

1.3 Aufbau des Papiers

Kapitel 2 grenzt das Design von verwandten Arbeiten ab. Kapitel 3–4 spezifizieren Architektur und Compute Layer. Kapitel 5 entwickelt die Tokenomik, Kapitel 6 das formale Verifikationsmodell. Kapitel 7 behandelt Training und Modellentwicklung, Kapitel 8 den Agent Layer, Kapitel 9 Vertraulichkeit und Risikomodell, Kapitel 10 Governance und Modell-Herkunft, Kapitel 11 offene Forschungsfragen. Anhang A enthält Kern-Datentypen und Referenzalgorithmen, Anhang B die Anreiz-Herleitungen.

2. Verwandte Arbeiten und Abgrenzung

Qubic (uPoW / Aigarth) [2]. Qubic ersetzt Hash-Rätsel durch das Training neuronaler Netze; Miner erzeugen Trainings-Solutions, die zugleich als Arbeitsnachweis für die Computor-Wahl dienen. Qubic ist der direkteste Vorläufer der Konsens-Kopplung in dieser Arbeit. Unterschiede: Qubic verifiziert *Trainings*-Beiträge, deren Nutzen statistisch bewertet wird, Myelith verifiziert *Inferenz*-Segmente objektiv gegen eine kanonische Referenz; Qubics Arbeitsprodukt ist ein evolvierendes Forschungssystem, Myeliths Arbeitsprodukt ist ein sofort nutzbarer Dienst, dessen Nachfrage den Coin-Kreislauf antreibt.

Bittensor (TAO) [3]. Bittensor ist ein Marktplatz von Subnetzen, in denen Validatoren Miner-Antworten qualitativ bewerten (Yuma-Konsens). Die Bewertung ist subjektiv-statistisch und hat wiederholt Gaming-Probleme gezeigt (Weight-Copying, Kollusion). Myelith vermeidet subjektive Bewertung vollständig: Ein Segment ist korrekt oder falsch, entscheidbar durch Hash-Gleichheit und (im Streitfall) durch eine kanonische Nachrechnung (Kap. 6). Es gibt keinen Bewertungsspielraum und keine Schwelle, an der sich ein Angreifer ausrichten könnte. Der Preis dafür: Myelith kann nur *ein* kanonisches Netzwerkmodell betreiben, keinen offenen Modell-Markt.

Petals [4]. Petals demonstriert BitTorrent-artiges Pipeline-Serving großer Open-Weight-Modelle über Freiwilligen-Hardware, der praktische Machbarkeitsnachweis für unseren Compute Layer. Petals hat jedoch weder Verifikation noch Anreize noch einen Ledger; die Nodes werden dort als ehrlich angenommen. Myelith lässt sich als "Petals + Verifikation + Konsens + Ökonomie" lesen.

Gensyn / Verde und RepOps [5][15]. Gensyn verifiziert dezentrale ML-Berechnung über *Refereed Delegation*:

Mehrere Anbieter rechnen dieselbe Aufgabe, und ein Streitspiel entscheidet, sobald sie sich widersprechen, korrekt, sofern mindestens einer ehrlich ist. Das Bisektions-Spiel in Kapitel 6.6 folgt derselben Tradition (Truebit, Arbitrum). Zentral für diese Arbeit ist Gensyns zweiter Beitrag: RepOps, eine Bibliothek reproduzierbarer Operatoren, die Hardware-Nichtdeterminismus durch feste Gleitkomma-Reihenfolgen beseitigt und damit bitweise Reproduzierbarkeit über verschiedene Hardware nachweist. RepOps belegt, dass bitweise Reproduzierbarkeit über Hardware-Grenzen hinweg erreichbar ist, und ist damit der nächstliegende Vergleichspunkt zu Kapitel 6.2. Myelith wählt dennoch einen anderen Weg: RepOps erzwingt Reproduzierbarkeit innerhalb der Gleitkommaarithmetik und bezahlt dafür mit eingeschränkter Parallelisierung; zudem deckt die Bibliothek ausdrücklich nur den Einzelgerätefall ab, während Reproduzierbarkeit über mehrere Knoten mit Pipeline- oder Tensor-Parallelismus als offene Arbeit benannt wird, also genau der Fall, der in Myelith vorliegt.

TOPLOC [14]. TOPLOC schlägt lokalitätssensitive Commitments über Zwischenaktivierungen vor, die Eingriffe an Modell, Prompt oder Präzision zuverlässig erkennen, dabei aber robust gegenüber unterschiedlichen GPU-Typen und algebraischen Umordnungen bleiben, bei sehr kompakter Beweisgröße und einer Prüfung, die schneller abläuft als die ursprüngliche Erzeugung. TOPLOC adressiert das Vertrauensproblem gegenüber einem *einzelnen* Inferenzanbieter und kennt weder Konsens noch Ökonomie noch Sharding. Myelith hat diesen Weg geprüft und verworfen: Toleranzbasierte Commitments tragen über verkettete Ausführung nicht und sind adaptiv angreifbar (Kap. 6.3). Sie bleiben jedoch der aussichtsreichste Kandidat, falls sich die ganzzahlige Ausführung als zu einschränkend erweisen sollte, und werden in Kapitel 10 als Forschungsrichtung geführt.

Frühe ganzzahlige Transformer-Inferenz. Die Linie beginnt vor den in Kapitel 6 herangezogenen Arbeiten. Dyadische Arithmetik wurde zunächst für eine reine Ganzzahl-Pipeline bei Faltungsnetzen entwickelt [42], ist jedoch auf lineare und stückweise lineare Operationen zugeschnitten und für die nichtlinearen Operationen in Transformern nicht anwendbar. Die erste Arbeit, die vollständige Ganzzahl-Inferenz für Transformer anstrebte, ersetzte die Quadratwurzel in der Normalisierung durch eine L1-Norm-Entsprechung [41]; I-BERT [18] folgte mit ganzzahligen Polynomapproximationen für Softmax, GELU und Layer Normalization. Myelith übernimmt aus dieser Linie das Ergebnis, nicht das Verfahren: Entscheidend ist für uns allein, dass eine vollständig ganzzahlige Ausführung möglich ist, da daraus der Determinismus folgt (Kap. 6.2).

VeriLLM. VeriLLM ist ein öffentlich verifizierbares dezentrales Inferenz-Framework auf Blockchain-Grundlage und teilt mit dieser Arbeit

mehrere Entwurfsziele: nachprüfbare Korrektheit, geringer Verifikations-Overhead, deterministische Zurechenbarkeit, Ununterscheidbarkeit der Aufgabentypen und Verträglichkeit mit heterogener Hardware [43]. Der Ansatz vermeidet einen Verifikationsengpass durch eine isomorphe Architektur, in der Inferenz- und Prüfrollen auf denselben Rechenknoten laufen; das erhöht die Auslastung und vergrößert zugleich die Menge möglicher Prüfer. Der Unterschied zu Myelith liegt im Zweck: VeriLLM sichert die Korrektheit von Inferenz, aber keinen Ledger. Die geleistete Arbeit trägt dort keinen Konsens, und es entsteht kein Wertkreislauf, in dem dieselbe Arbeit Vergütung und Stimmgewicht bestimmt. Die isomorphe Rollenverteilung ist gleichwohl eine ernstzunehmende Alternative zur Redundanz mit $r = 2$ und wird in Kapitel 11, Punkt 18, als solche geführt.

Softwareseitige Gleitkomma-Emulation. Neben der Reihenfolgenvorschrift [15] und der Toleranzprüfung [14] existiert ein dritter Weg zu plattformübergreifend gleichen Ergebnissen: Gleitkommaoperationen vollständig in Software zu emulieren statt die Recheneinheiten der Hardware zu nutzen. Optimistische und abstimmungsbasierte Verfahren setzen diesen Weg voraus, um über verschiedene Plattformen hinweg konsistente Ergebnisse zu erhalten [9]. Er ist bitgenau und stellt keine Anforderungen an die Hardware, verlangsamt die Inferenz jedoch erheblich, da jede einzelne Operation aus Ganzzahlbefehlen zusammengesetzt wird. Myelith verfolgt ihn nicht: Wenn ohnehin ganzzahlig gerechnet wird, ist es folgerichtig, das Modell selbst ganzzahlig auszuführen, statt Gleitkomma über Ganzzahlen nachzubilden. Der Unterschied ist erheblich – im ersten Fall entspricht eine Modelloperation einer Maschinenoperation, im zweiten einer Folge von Dutzenden.

Ganzzahlige Inferenz (I-BERT, I-ViT, I-LLM) [18][19][20]. Diese Arbeiten verfolgen ein anderes Ziel als Myelith: Sie quantisieren Transformer vollständig auf Ganzzahlarithmetik, um Speicherbedarf, Latenz und Energieverbrauch zu senken, insbesondere für Endgeräte [18][19][20]. Determinismus ist dort kein Entwurfsziel, sondern ein Nebenprodukt. Genau dieses Nebenprodukt ist für Myelith konstitutiv: Da Ganzzahladdition assoziativ ist, liefert eine so ausgeführte Inferenz bitgleiche Ergebnisse ohne jede Vorschrift über die Reihenfolge der Operationen (Kap. 6.2). Myelith trägt hier keinen quantisierungstechnischen Beitrag bei, sondern die Beobachtung, dass ganzzahlige Ausführung die Verifikationsfrage verteilter Inferenz an der Wurzel löst, sowie den Nachweis der dafür nötigen Zusatzfestlegungen (Anhang B.5).

Numerisches Verhalten von Matrixeinheiten [21]. Untersuchungen der auf KI ausgelegten Matrixmultiplizierer zeigen, dass diese gegenwärtig nicht dem IEEE-754-Verhalten entsprechen und sich zwischen Herstellern in Rundungsverhalten, Akkumulatorbreite und Normalisierungspunkten unterscheiden, mit nicht

reproduzierbaren Ergebnissen über Architekturgenerationen hinweg [21]. Dieser Befund ist der Grund, weshalb Myelith den gleitkommabasierten Weg nicht beschreitet: Eine Reihenfolgenvorschrift setzt einheitliches Rundungsverhalten der einzelnen Instruktion voraus, und diese Voraussetzung ist für die schnellen Rechenpfade moderner Beschleuniger nicht erfüllt.

Ora (opML) und zkML-Systeme (EZKL, Modulus) [6]. Optimistische bzw. Zero-Knowledge-Verifikation einzelner ML-Inferenzam sind etabliert; beide dienen dort aber als Orakel für bestehende Chains. Myelith invertiert das Verhältnis: Die Inferenz ist nicht Gast auf einer Chain, sondern deren Arbeitsgrundlage.

HadAgent [1]. HadAgent prägt den Begriff Proof-of-Inference für einen Konsens, in dem Nodes Blockrechte durch deterministische LLM-Inferenz verdienen. Das ist die direkteste Vorarbeit zu Kapitel 3.5, terminologisch wie konzeptionell. Der zentrale Unterschied liegt in der Modellgröße und damit der gesamten Verifikationsmechanik: HadAgent verifiziert per Voll-Nachrechnung eines einzelnen Forward-Passes durch Master-Nodes einer Zwei-Klassen-Architektur, wobei das Modell vollständig auf einen Node passen muss. Myelith adressiert Modelle, die kein einzelner Node halten kann: Pipeline-Sharding über Pods, Verifikation per Redundanz plus Bisektion (Streitbeilegung durch einen einzigen Shard-Forward statt Voll-Nachrechnung), arbeitsgewichtete Validatoren-Wahl und ein Burn-and-Mint-Kreislauf, der Coin und Inferenz-Zugang koppelt.

Proof of Quality (PoQ) und PolyLink [7][8]. PoQ ersetzt Berechnungsverifikation durch Output-Qualitätsbewertung mittels leichter Evaluationsmodelle; PolyLink kombiniert VRF-gewählte Validatoren-Komitees mit LLM-as-a-Judge-Scoring. Beide akzeptieren subjektive Bewertung als Preis für Geschwindigkeit; Myelith bleibt bei objektiver Entscheidbarkeit (vgl. die Bittensor-Abgrenzung).

Blockchain-gestütztes föderiertes Lernen. Ein umfangreicher Forschungszweig verbindet Blockchains mit föderiertem Lernen, um Herkunft und Integrität von Trainingsbeiträgen nachprüfbar zu machen. Vorgeschlagen wurden unter anderem Merkle-basierte Provenienz von Datenpunkten und Updates mit kompakten On-Chain-Metadaten [30], Zero-Knowledge-Beweise für lokale Trainingsschritte einschließlich Vorwärts- und Rückwärtspass [31] sowie konsensgestützte Verfahren zur Erkennung vergifteter Beiträge [32][33]. Die Datenprovenienz aus Kapitel 7.3 steht in dieser Linie und beansprucht keine Neuheit für das Verfahren als solches. Der Unterschied liegt im Kontext: Die genannten Arbeiten sichern föderiertes Lernen zwischen wenigen, meist bekannten Organisationen ab, häufig auf zugangsbeschränkten Blockchains. Myelith arbeitet mit anonymen, wirtschaftlich motivierten Teilnehmern in einem offenen

Netz, in dem dieselbe Infrastruktur zugleich Inferenz erbringt und den Konsens trägt.

Byzantinisch robuste Aggregation. Die Verfahren, auf die sich Kapitel 7.4 stützt, stammen aus dieser Literatur: Krum wählt das Update mit der geringsten Distanzsumme zu seinen Nachbarn [34], koordinatenweiser Median und getrimmter Mittelwert ersetzen den Mittelwert durch robuste Ordnungsstatistiken [35], Bulyan kombiniert beide Ansätze. Belegt ist dort auch die Eigenschaft, auf der Kapitel 7.4 beruht: Koordinatenweise Verfahren tolerieren bis zur Hälfte byzantinischer Beiträge [35]. Myelith übernimmt den Median unverändert; der eigene Beitrag beschränkt sich auf die Beobachtung, dass er in ganzzahliger Arithmetik nur Vergleiche benötigt und damit im Verifikationsmodell aus Kapitel 6 nachprüfbar bleibt. Zu beachten ist die bekannte Grenze dieser Verfahren: Bei nicht identisch verteilten Daten und bei Angreifermehrheiten verlieren sie ihre Garantie.

Abwehr eingeschleuster Anweisungen. Der Agent Layer (Kap. 8.3) stützt sich auf das Dual-LLM-Muster [39] und dessen Ausarbeitung in CaMel [40], die Kontroll- und Datenfluss architektonisch trennen, sodass abgerufene Inhalte den Ablauf nicht beeinflussen können. Myelith beansprucht hier keine Neuheit. Der Beitrag liegt in der Verbindung mit dem Konsens: Da Budget, Empfängerliste und Zeitfenster im Session-Kontrakt und damit außerhalb des Modellkontexts liegen, wird die Durchsetzung nicht von einer Softwareschicht übernommen, sondern von der Kette selbst. Ein eingeschleuster Text kann die Grenzen daher auch dann nicht verschieben, wenn die Trennung im Modell versagt.

DeServe [9]. DeServe untersucht, wie sich Inferenz großer Modelle durch Dezentralisierung verbilligen lässt, und ist damit dem Compute Layer dieser Arbeit verwandt. Verifikation und Konsens sind dort nicht Gegenstand; der Beitrag liegt auf der Kostenseite.

Render, io.net, Akash (DePIN-Compute). Diese Netzwerke vermitteln GPU-Kapazität als Rohstoff; die Arbeit sichert keinen Konsens, und es gibt kein kanonisches Modell. Sie sind Marktplätze; Myelith ist ein Organismus: ein Modell, ein Ledger, ein Kreislauf.

Zusammenfassende Abgrenzung: Der Begriff Proof of Inference und die Grundidee der Inferenz als Konsensarbeit sind durch HadAgent vorweggenommen und werden hier nicht als neu beansprucht. Kein existierendes System vereint jedoch (a) ein einzelnes großes, pipeline-verteiltes agentisches Modell, (b) objektiv verifizierte Inferenz als Konsens-relevanten Arbeitsnachweis, deren Determinismus aus der Arithmetik selbst folgt statt aus einer Ausführungsvorschrift und (c) einen Burn-and-Mint-Kreislauf, in dem der geminte Coin das Zugangsrecht zum Arbeitsprodukt ist. Diese Kombination ist der Beitrag dieser Arbeit.

3. Architektur

3.1 Designziele und Grundannahmen

Das Netzwerk verfolgt drei gleichzeitige Ziele, die klassische Blockchains und dezentrale Compute-Netzwerke bisher getrennt lösen:

1. **Ledger-Sicherheit:** Ein manipulationssicheres, dezentrales Transaktionsregister.
2. **Nutzbare Rechenleistung:** Die für die Sicherheit aufgewendete Rechenarbeit betreibt ein großes agentisches Sprachmodell (im Folgenden: *das Netzwerkmodell*), statt Hashes zu verbrennen.
3. **Geschlossener Wertkreislauf:** Die durch Mining erzeugte Währung ist zugleich das Zahlungsmittel für Inferenz auf dem Netzwerkmodell (Burn-and-Mint-Ökonomie).

Zentrale Annahmen:

- Teilnehmer-Hardware ist heterogen (Consumer-GPUs bis Rechenzentrums-Cluster) und über normale Internetverbindungen angebunden (Latenz 20–200 ms, Bandbreite 50 Mbit/s – 10 Gbit/s).
- Teilnehmer sind rational-ökonomisch und potenziell byzantinisch (bis zu einem Anteil $f < 1/3$ der gewichteten Stimmen).
- Das Netzwerkmodell ist zu groß für einzelne Nodes (Zielgröße: 100B–1T+ Parameter) und muss daher über mehrere Nodes verteilt (sharded) werden.

3.2 Schichtenmodell

Die Architektur trennt strikt zwischen Konsens und Compute, koppelt beide aber über kryptografische Arbeitsnachweise:



Abbildung 1: Schichtenmodell. Proof of Inference koppelt Compute- und Consensus-Layer.

Kernentscheidung: Der Konsens (L1) läuft *nicht* auf den Inferenz-Ergebnissen selbst, sondern auf kompakten, verifizierbaren *Arbeitsnachweisen* (Proof of Inference, PoI), die der Compute Layer produziert. Damit bleibt die Blockzeit unabhängig von der Inferenz-Latenz.

3.3 Netzwerk-Rollen

Rolle	Aufgabe	Hardware	Anreiz
Shard-Miner	Halten je einen Modell-Shard (zusammenhängende Layer-Gruppe) im VRAM und berechnen Forward-Passes	GPU ≥ 16–24 GB VRAM	Block-Reward anteilig zur nachgewiesenen Inferenzarbeit
Pod-Koordinator	Gewählter Miner eines Pods; orchestriert die Pipeline, sammelt Teilnachweise, reicht aggregierten PoI ein	wie Shard-Miner + gute Anbindung	Koordinations-Bonus
Validatoren	Führen den BFT-Konsens aus, prüfen PoI-Stichproben, verwalten Stake/Slashing	CPU-lastig, 1 GPU für Spot-Checks	Anteil an Gebühren + Inflation-Reward
Checker (Fisherman)	Rechnen zufällig ausgewählte Inferenz-Segmente nach und melden Abweichungen	beliebige GPU	Kopfgeld aus geslashtem Stake
Gateways	Nehmen Nutzeranfragen entgegen, routen zu Pods, liefern Streams zurück	Netzwerk-lastig	Anteil der Inferenzgebühr
Nutzer	Verbrennen Coins für Inferenz-Credits	–	Zugriff auf das Netzwerkmodell

3.4 Verifikation im Überblick

Das teuerste Problem des Systems ist der Nachweis korrekter Berechnung. Die Architektur kombiniert drei Mechanismen mit unterschiedlichem Kosten-Sicherheits-Profil:

Stufe 1, Deterministische Redundanz (sofort, günstig):

Jedes Inferenz-Segment wird von $r = 2$ unabhängig zugelosten Pods parallel berechnet. Da die Ausführung vollständig ganzzahlig erfolgt und Ganzzahladdition assoziativ ist, sind die Ergebnisse bitidentisch, gleichgültig wie die jeweilige Hardware parallelisiert; verglichen werden schlichte Commitment-Hashes. Der Vergleich ist binär und ohne Parameter (Kap. 6.2). Redundanzfaktor 2 kostet 50 % Effizienz. Das ist der Preis der Dezentralität und geht als expliziter Posten in die Ökonomie ein.

Stufe 2, Optimistische Stichproben (verzögert, gezielt):

Checker rechnen VRF-ausgeloste Segmente (~1–3 % des Volumens) vollständig nach. Bei Abweichung startet ein **Bisektions-Spiel** (analog Truebit [10] und Arbitrum [11]): Der Streit wird binär auf den ersten abweichenden Übergang eingegrenzt; nur dieser eine Shard-Forward wird von den Validatoren on-chain nachgerechnet. Da das korrekte Ergebnis kanonisch ist, ist die Schuldzuweisung eindeutig. Der Verlierer wird geslasht, der Checker erhält ein Kopfgeld.

Stufe 3, zkML-Anker (selten, teuer, maximal sicher):

Für besonders wertvolle Ergebnisse (z. B. Abschlüsse agentischer Transaktionen mit Finanzwirkung) können Nutzer gegen Aufpreis einen Zero-Knowledge-Beweis der Inferenz anfordern. zk-Beweise für vollständige LLM-Forward-Passes sind heute noch um Größenordnungen zu teuer für den Regelbetrieb; die Architektur sieht sie als optionalen Premium-Pfad und als Aufrüstpfad vor, sobald zkML-Systeme effizient genug werden.

Determinismus ohne Ausführungsvorschrift: Verbindlich sind das Quantisierungsschema und wenige arithmetische Festlegungen, nicht aber Reihenfolge, Blockaufteilung oder Kernel-Implementierung (Kap. 6.2). Heterogene Hardware nimmt damit ohne Durchsatzverlust und ohne Wettbewerbsnachteil teil.

Ökonomische Sicherung: Shard-Miner hinterlegen Stake proportional zu ihrer Reward-Kapazität. Die erwartete Strafe (Slash-Wahrscheinlichkeit $\times$ Stake) muss den erwarteten Gewinn aus Falschberechnung übersteigen; die Stichprobenrate ist der Stellhebel und wird per Governance an die beobachtete Betrugsrate angepasst.

3.5 Consensus Layer: Proof of Inference + BFT

3.5.1 Warum kein reines "Inference-PoW"

Leader-Wahl direkt über Inferenz-Wettrennen (wer zuerst rechnet, schreibt den Block) wäre manipulierbar (Grinding über Eingaben) und würde Blockzeit an Inferenz-Latenz koppeln. Stattdessen:

3.5.2 Zwei entkoppelte Prozesse

Prozess A, Blockproduktion (schnell):

Ein Komitee von Validatoren (gewählt nach Stake, rotierend per VRF) führt einen klassischen BFT-Konsens (HotStuff-Familie [12]) mit Blockzeiten von 1–2 s aus. Blöcke enthalten: Transaktionen, Inferenz-Aufträge (als Commitments), aggregierte PoI-Nachweise, Slashing-Events.

Prozess B, Arbeitsnachweis (kontinuierlich):

Pods reichen pro Epoche signierte PoI-Bündel ein: Merkle-Wurzeln über (Eingabe-Commitment, Ausgabe-Commitment, Segment-Metadaten, Signaturen aller beteiligten Shard-Miner). Der Block-Reward einer Epoche wird proportional zur *bestätigten* Inferenzarbeit (nach Stufe-1-Übereinstimmung, abzüglich später widerlegter Segmente) auf die Miner verteilt.

Kopplung: Das Stimmgewicht der Validatoren-Wahl speist sich aus zwei Quellen: gestaktem Coin und nachgewiesener historischer Inferenzarbeit (mit Abklingfaktor). Damit sichert die nützliche Arbeit indirekt den Konsens. Wer das Netzwerk angreifen will, muss entweder massiv Coins kaufen (Markt-Feedback) oder dauerhaft ehrliche Inferenzarbeit leisten (Selbstwiderspruch).

3.5.3 Datenverfügbarkeit

Vollständige Prompts/Ausgaben gehören nicht on-chain (Datenschutz, Volumen). On-chain stehen nur Commitments; die Rohdaten liegen verschlüsselt beim Nutzer und (für die Streitfrist von z. B. 7 Tagen) als Erasure-codierte Fragmente bei den beteiligten Pods, damit Bisektions-Spiele durchführbar bleiben.

4. Compute Layer: Modell-Sharding und Pods

4.1 Pipeline-Parallelismus als Grundmuster

Tensor-Parallelismus scheidet über WAN aus (All-Reduce pro Layer benötigt Sub-Millisekunden-Latenz). Die Architektur setzt deshalb auf **Pipeline-Parallelismus**: Das Modell wird in k zusammenhängende Shards zerlegt (z. B. Layer 1–20, 21–40, ...). Ein **Pod** ist eine Kette aus k Shard-Minern, die gemeinsam einen vollständigen Forward-Pass ausführen. Zwischen den Shards fließen nur Aktivierungen (bei Batch 1 und hidden dim 8192 in fp8: ~8 KB pro Token pro Übergabe), das ist WAN-tauglich.

Die Wahl von k ist ein Sicherheitsparameter. Es liegt nahe, k klein zu wählen: weniger, größere Shards bedeuten weniger Netzübergänge und damit geringere Pipeline-Latenz. Diese Optimierung ist jedoch nicht kostenlos, denn k steuert gleichzeitig drei Größen in unterschiedliche Richtungen:

1. **Latenz.** Je kleiner k , desto weniger Übergänge, desto schneller die Pipeline (spricht für kleines k).
2. **Kollusionsresistenz.** Die Wahrscheinlichkeit, dass zwei redundante Pods gemeinsam ein falsches Ergebnis durchbringen, fällt exponentiell mit k ($P_{\text{koll}} \approx \beta^{2k}$, Anhang B.2). Halbiert man k , quadriert sich diese Wahrscheinlichkeit (spricht für großes k).
3. **Zugangsschwelle und Dezentralität,** größere Shards erfordern mehr VRAM pro Knoten. Überschreitet der Shard den VRAM verbreiteter Consumer-Karten, sind nur noch Rechenzentren teilnahmefähig, und Consumer-Hardware fällt aus dem Netz. Damit steigt β , was Wirkung (2) zusätzlich verstärkt (spricht für großes k).

k ist deshalb pro Modellversion konfigurierbar und unterliegt der Governance (Kap. 10.3), nicht der Optimierung durch einzelne Betreiber. Die Standardwahl $k = 8$ orientiert sich am VRAM verbreiteter Consumer-GPUs; eine Absenkung ist nur vertretbar, wenn die resultierende Kollusionsschranke explizit nachgerechnet und die Zugangsschwelle bewertet wird.

Latenz-Topologie: Der Networking Layer misst kontinuierlich Paarlatenzen (Pings über Gossip). Der Pod-Bildungs-Algorithmus (deterministisch aus dem Blockhash + Latenzgraph, siehe 4.3) gruppiert bevorzugt geografisch nahe Miner in denselben Pod, um die Pipeline-Latenz zu minimieren, während die Shard-Zuteilung *innerhalb* eines Pods zufällig bleibt (Kollusionsschutz).

4.2 Durchsatz statt Einzellatenz

Ein einzelner Forward-Pass durch einen WAN-Pod ist langsam (Größenordnung 0,5–2 s Pipeline-Latenz plus Rechenzeit pro Token). Die Architektur kompensiert das durch:

- **Micro-Batching / Pipelining:** Während Shard 3 Token t berechnet, verarbeitet Shard 1 bereits Token $t+2$. Bei kontinuierlichen Streams (agentische Sessions) nähert sich der Durchsatz dem eines einzelnen Shards an.
- **KV-Cache-Lokalität:** Der KV-Cache jeder Session bleibt auf den Shards des zugewiesenen Pods (Session-Affinität). Pod-Wechsel erfordern Cache-Rebuild und werden nur bei Ausfall oder Epochenwechsel ausgelöst.
- **Spekulatives Decoding:** Kleine Draft-Modelle, die vollständig auf einzelne Miner passen, generieren Token-Kandidaten, die der Pod in einem einzigen batched Forward-Pass verifiziert, reduziert die Zahl der teuren Pipeline-Durchläufe um Faktor 2–4.

4.3 Epochen und deterministische Zuteilung

Die Zeit ist in **Epochen** (z. B. 1 Stunde) unterteilt. Zu Epochenbeginn wird aus dem finalisierten Blockhash der Vorepoche ein Seed abgeleitet, der über eine verifizierbare Zufallsfunktion (VRF) bestimmt:

1. welche registrierten Miner welchen Shard erhalten,
2. wie Pods zusammengesetzt werden (unter Latenz-Nebenbedingungen),
3. welche Inferenz-Segmente in dieser Epoche der Stichprobenprüfung unterliegen.

Da die Zuteilung deterministisch aus öffentlichen Daten folgt, kann jeder Teilnehmer sie unabhängig nachvollziehen, es gibt keine zentrale Scheduling-Instanz.

4.4 Nähe innerhalb, Distanz zwischen den Pods

Latenz und Kollusionsresistenz ziehen die Pod-Bildung in entgegengesetzte Richtungen. Die Auflösung liegt darin, beide Ziele auf *verschiedenen Ebenen* zu verfolgen:

- **Innerhalb eines Pods** wird auf Nähe optimiert: Die Mitglieder einer Pipeline sollen möglichst geringe Paarlatenzen aufweisen (Kap. 4.1), denn hier addieren sich Verzögerungen über k Übergänge.
- **Zwischen den beiden redundanten Pods** wird Distanz *erzwungen*: Der Zwillings-Pod eines Segments muss aus einer anderen geografischen Zone stammen, mit Diversitätsanforderungen auch bezüglich autonomer Systeme (AS) und, soweit ermittelbar, Betreiber.

Die Begründung ist keine Latenzfrage, sondern die Grundlage der Stufe-1-Sicherheit: Redundanz schützt nur, wenn die beiden Berechnungen *unabhängig* sind. Zwei Pods im selben Rechenzentrum, unter derselben Jurisdiktion oder am selben Stromnetz sind korrelierte Ausfall- und Kollusionsrisiken, die Annahme unabhängiger Fehler (Anhang B.2) wäre verletzt, und ein einzelner rechtlicher Zugriff könnte beide Berechnungen zugleich beeinflussen. Der Preis ist gering: Da die redundanten Pods nicht miteinander kommunizieren, sondern nur ihre Commitments einreichen, kostet ihre Entfernung keine Pipeline-Latenz, sondern verzögert

lediglich den Zeitpunkt des Stufe-1-Abgleichs.

5. Tokenomics – Der Burn-and-Mint-Kreislauf

5.1 Grundprinzip

Der native Coin MYL erfüllt drei Funktionen: Sicherung des Konsenses (Staking), Vergütung der Miner (Minting) und Bezahlung der Inferenz (Burning). Der Kreislauf ist geschlossen:

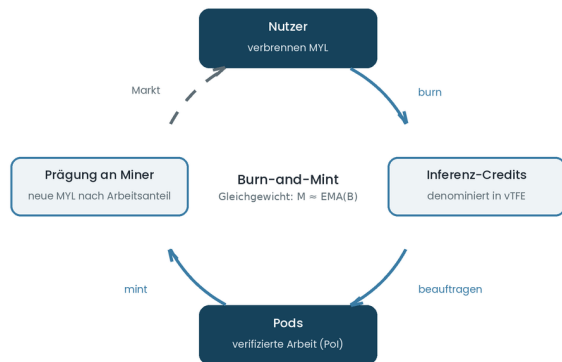


Abbildung 2: Der Burn-and-Mint-Kreislauf. Durchgezogen die Protokollvorgänge, gestrichelt der Markt.

Training fügt diesem Kreislauf keine zweite Geldquelle hinzu, sondern eine Rückkopplung: Verbessert sich die Modellqualität, steigt die Nachfrage und damit der Burn, aus dem sich die Prägung speist. Die Ausgabe für Training rechtfertigt sich also über die Nachfrage, die sie erzeugt, nicht über die aufgewandte Rechenzeit (Anhang B.7).

Das Protokoll gibt MYL ausschließlich an Miner aus; Nutzer erwerben sie am Markt. Prägung und Verbrennung sind Protokollvorgänge, der Erwerb ist es nicht.

Inferenz-Credits sind in **Rechenarbeit** denominiert (Einheit: verifizierte Token-Forward-Äquivalente, *vTFE*), nicht in Fiat oder MYL. Damit ist der Nutzpreis der Inferenz stabil in Recheneinheiten; der MYL-Preis vermittelt zwischen Angebot (Miner-Kapazität) und Nachfrage (Credit-Käufe).

5.2 Die Prägefunktion

Sei B_e das in Epoche e verbrannte MYL-Volumen und w_e die bestätigte Arbeit (in *vTFE*). Die Prägung M_e der Epoche:

$$M_e = \min(B_e \cdot (1 + s), M_{\max})$$

Dabei bezeichnet M_e die Prägung der Epoche e , B_e das exponentiell geglättete Burn-Volumen, s die Subventionsrate und $M_{\max}$ den Emissionsdeckel.

mit:

- B_e = exponentiell geglättetes Burn-Volumen (EMA über ~30 Epochen, dämpft Manipulation durch Burn-Spikes),
- s = Subventionsrate (Bootstrap-Parameter, startet z. B. bei 0,5 und fällt per Governance-Schedule gegen 0),
- $M_{\max}$ = harter Inflations-Deckel pro Epoche (Restemission aus fixem Gesamtangebot, analog Halving-Schedule).

Eigenschaften:

1. Im Gleichgewicht ($s \rightarrow 0$) gilt $M_e \approx B_e$: Die Geldmenge ist langfristig **netto-neutral bis deflationär** (verbrannte Coins $\geq$ geprägte, da Slashing-Burns hinzukommen).
2. In der Bootstrap-Phase ($s > 0$) subventioniert Inflation den Aufbau von Miner-Kapazität, bevor Nachfrage existiert, dieselbe Logik wie die Block-Subvention in Bitcoin [13] vor relevanten Gebühren.

5.3 Verteilung der Prägung

M_e wird aufgeteilt:

78 %	Shard-Miner	(proportional zu bestätigten <i>vTFE</i> , nach Redundanz-Normierung)
5 %	Pod-Koordinatoren	(proportional zu koordinierten Segmenten)
10 %	Validatoren	(proportional zu Stake $\times$ Uptime)
4 %	Checker-Pool	(Grundvergütung; Kopfgeider kommen zusätzlich aus Slashes)
3 %	Protokoll-Treasury	(Governance-verwaltet: Modell-Updates, Audits)

Trainingsvergütung. Trainingsarbeit (Kap. 7) wird nicht aus der Prägung finanziert, sondern aus der Protokoll-Treasury und einem per Governance abschaltbaren Aufschlag auf die Inferenzgebühr. Diese Wahl ist nicht beliebig: Eine Finanzierung aus Zusatzprägung würde die Netto-Inflation nahezu verdoppeln und damit alle Halter verwässern, während Treasury und Gebührenaufschlag den Kreislauf unberührt lassen (Anhang B.7).

Für die Vergütung je Rechenstunde gilt eine Obergrenze: Sie darf die Inferenzvergütung nicht erreichen. Andernfalls verlagern Miner Kapazität von der Inferenz auf das Training und entziehen dem Netzwerk seine einzige Einnahmequelle. Der Entwurf setzt höchstens 70 Prozent der Inferenzvergütung an; der Wert ist ein Governance-Parameter (Kap. 10.3).

Redundanz-Normierung: Da jedes Segment von $r = 2$ Pods berechnet wird, erhält jeder Pod die halbe *vTFE*-Gutschrift. Miner werden also für *nützliche Netto-Arbeit* bezahlt; der Redundanz-Overhead ist eingepreist, nicht versteckt.

5.4 Credit-Preisbildung

Der Umtauschkurs MYL $\rightarrow$ *vTFE* wird pro Epoche algorithmisch gestellt (EIP-1559-analog):

$$P_{e+1} = P_e \cdot \exp(\kappa (u_e - u^*))$$

Hier ist P_e der Credit-Preis der Epoche e , u_e die gemessene Auslastung, u^* das Auslastungsziel und κ eine Dämpfungskonstante.

- $\text{auslastung}_e = \text{nachgefragte vTFE} / \text{verfügbare Pod-Kapazität}$,
- $\text{ziel} = 0,8$ (Puffer für Lastspitzen),
- $k = \text{Dämpfungskonstante}$.

Bei Überlast steigt der Credit-Preis → Nachfrage sinkt, Mining wird attraktiver → Kapazität wächst. Der Mechanismus ersetzt zentrales Kapazitätsmanagement durch Preissignale.

5.5 Staking und Slashing-Matrix

Akteur	Stake-Pflicht	Slash-Grund	Slash-Höhe
Shard-Miner	$\propto$ beanspruchte Reward-Kapazität	falsches Ergebnis (per Bisektion bewiesen)	100 % Stake
Shard-Miner	–	Nichtverfügbarkeit während Session	1–5 % (gestaffelt)
Pod-Koordinator	Zusatz-Stake	falsche Pol-Aggregation	100 %
Validator	BFT-Stake	Double-Signing / Zensur (bewiesen)	30–100 %
Checker	Kautio pro Anfechtung	mutwillig falsche Anfechtung	Kautio

Trainingssegmente. Für sie gilt dieselbe Struktur, jedoch mit anderer Gewichtung: Der Gewinn aus Betrug ist geringer, da die Trainingsvergütung niedriger liegt, der Schaden dagegen größer, denn ein durchgerutschtes Inferenz-Segment betrifft eine Antwort, ein durchgerutschter Gradient hingegen das Modell und damit alle künftigen Antworten. Angehoben wird deshalb nicht der Stake, sondern die **Stichprobenrate**: Sie wirkt unmittelbar und kostet Kapazität statt Kapitalbindung.

Anreiz-Ungleichung (Sicherheitsbedingung): Ein Miner betrügt rational nur, wenn erwarteter Gewinn $>$ erwartete Strafe. Mit Stichprobenrate p (Stufe 2), Stake s und Betrugsgewinn pro Segment g :

$$p \cdot s > g/p \iff S_{\min} = g/p^2$$

Dabei ist p die Stichprobenrate, s der hinterlegte Stake und g der Gewinn aus einem betrogenen Segment. Die Herleitung steht in Anhang B.1.

Bei $p = 2\%$ und $g = \text{Reward eines Segments}$ folgt $S_{\min} = 2500$ Segment-Rewards, der Stake entspricht damit etwa zwölf Epochen-Einkommen. Das ist der quantitative Anker für die Stake-Parameter und wird per Governance an gemessene Betrugsraten angepasst.

5.6 Angriffsvektoren und Gegenmaßnahmen

- **Self-Dealing (Miner kauft eigene Inferenz, um Prägung zu ernten):** Unrentabel per Konstruktion, solange $M_e \leq B_e$ (Gleichgewicht): Der Angreifer verbrennt mehr, als er zurückerhält (er bekommt nur seinen Kapazitätsanteil der Prägung). In der Subventionsphase ($s > 0$) wird Self-Dealing durch die EMA-Glättung und ein Burn-Cap pro Adresse gedämpft.
- **Grinding der Pod-Zulassung:** VRF-Seed stammt aus finalisiertem Block der Vorepoche; Miner-Registrierung schließt 2 Epochen vor Zuteilung

(kein Last-Minute-Einbringen präparierter Identitäten).

- **Sybil auf Checker-Kopfgeider:** Anfechtung kostet Kautio; falsche Anfechtungen verbrennen sie.

5.7 Ausgabestruktur und Anlaufphase

Kapitel 5.2 beschreibt, wie Prägung entsteht, sagt aber nichts darüber, wie das Netzwerk beginnt. Diese Lücke wird hier geschlossen.

Warum ein Start bei null nicht möglich ist. Der Entwurf sähe einen Fair Launch ohne jede Vorabmenge vor: Alle MYL entstehen aus verifizierter Arbeit. Das scheitert an einer Rückkopplung, die für einsatzbasierte Systeme seit langem beschrieben ist [36][37]: Ein Protokoll benötigt ein werthaltiges natives Asset, um gesichert zu sein, und muss gesichert sein, damit das Asset Wert erlangt. Arbeitsbasierte Systeme umgehen dies, indem sie knappe äußere Ressourcen in Coins überführen; in Myelith ist die Arbeit jedoch selbst an vorhandenen Einsatz gebunden. Miner müssen Stake hinterlegen, um überhaupt Arbeit annehmen zu dürfen (Anhang B.1); ohne vorhandene MYL kann niemand Stake stellen, ohne Miner entsteht keine Prägung, aus der Stake gebildet werden könnte. Der Stake-Bedarf übersteigt dabei den Credit-Bedarf der ersten Nutzer um mehr als das Hundertfache und bestimmt damit allein die erforderliche Anfangsmenge (Anhang B.8).

Wie die Anfangsmenge klein bleibt. Die Sicherheitsbedingung lautet $S_{\min} = g/p^2$, hängt also quadratisch von der Stichprobenrate ab. Wird p in der Anlaufphase erhöht, sinkt der Stake-Bedarf drastisch: Bei einer Rate von fünfzig Prozent statt zwei Prozent fällt er auf ein Sechshundertstel. Das kostet Kapazität, da jedes zweite Segment nachgerechnet wird, ist in einer Phase mit Überkapazität jedoch tragbar. Die Rate wird mit wachsendem Netz planmäßig auf den Zielwert gesenkt, während der Stake-Bedarf entsprechend steigt und aus der laufenden Prägung gedeckt werden kann.

Die Anfangsmenge bemisst sich damit am Stake-Bedarf der Anlaufphase unter erhöhter Prüfrate, nicht an einem gesetzten Zielwert.

Verteilung. Die Genesis-Menge geht ausschließlich an Teilnehmer des vorgelagerten Testnetzes, bemessen nach dort geleisteter und geprüfter Arbeit, zuzüglich des Treasury-Anteils aus Kapitel 5.3. Es findet kein Vorverkauf statt, und es gibt keine Zuteilung an Entwickler oder Investoren über die Treasury hinaus. Diese Festlegung folgt nicht nur aus dem Selbstverständnis des Protokolls: Eine Ausgabe gegen Zahlung mit Renditeerwartung wäre in vielen Rechtsordnungen anders zu bewerten als ein arbeitsgebundener Erwerb.

Kein fester Emissionsdeckel. Es liegt nahe, ein Gesamtangebot festzuschreiben oder die Prägung je Epoche zu deckeln. Beides ist hier nicht vorgesehen, und zwar aus einem Grund, der sich aus der Konstruktion ergibt: Die Prägung ist an den geglätteten Burn

gekoppelt und wächst daher ohnehin nur mit der Nachfrage. Ein zusätzlicher Deckel entkoppelt sie davon. Sobald die Nachfrage ihn überschreitet, wird geleistete Arbeit nicht mehr vollständig vergütet; Miner verlassen das Netz, und die Kapazität sinkt. Modellrechnungen zeigen, dass ein bindender Deckel den Umlauf nicht stabilisiert, sondern zum Erliegen bringt, da mehr verbrannt als geprägt wird (Anhang B.8). Die Knappheit entsteht in diesem System nicht aus einer Obergrenze, sondern aus der Kopplung an tatsächliche Nutzung.

Konzentration der Frühphase. Wer früh teilnimmt, erwirbt einen überproportionalen Anteil des insgesamt Geprägten. Das ist bei arbeitsgebundener Ausgabe unvermeidlich und für Fair Launches dokumentiert: Auch ohne Vorverkauf und Insider-Zuteilung können frühe Miner erhebliche Bestände ansammeln [38]. Das Gegenmittel ist nicht eine abweichende Verteilung, sondern eine flache Subventionskurve: Je niedriger die Startsubvention s , desto geringer der Vorteil früher Teilnahme. Der Parameter unterliegt der Governance (Kap. 10.3) und der Invariante aus Anhang B.4.

6. Verifikation — Formales Modell

Dieses Kapitel ersetzt das Verifikationsmodell aus v0.1. Die Änderung betrifft nicht das Prinzip, sondern seine Grundlage: v0.1 verlangte bitidentische Gleitkomma-Ausführung und musste dafür die Reihenfolge aller Operationen vorschreiben. v0.2 verlagert die Anforderung eine Ebene tiefer und führt die Inferenz **vollständig in Ganzzahlarithmetik** aus. Da die Ganzzahladdition assoziativ ist, entsteht Bitgleichheit dann ohne jede Reihenfolgenvorschrift, unabhängig davon, wie die Hardware parallelisiert. Abschnitt 6.3 dokumentiert, warum die naheliegenden Alternativen (Gleitkomma mit fester Reihenfolge, Toleranzvergleich) verworfen wurden.

6.1 Notation

Ein **Inferenz-Segment** σ ist ein Tupel (x, θ_v, π, y) :

- x = Eingabe-Commitment (Hash über Prompt-Chunk $\parallel$ KV-Cache-Wurzel),
- θ_v = Modellversion: Gewichte, Quantisierungsschema und Ausführungsspezifikation (6.5),
- π = Pipeline-Pfad (geordnete Miner-Liste des Pods),
- y = Ausgabe-Commitment.

Jeder Shard-Miner i im Pfad signiert seinen Übergang: $\text{sig}_i(h(a_{i-1}) \parallel h(a_i) \parallel \sigma_{id})$, wobei a_i die Aktivierungen nach Shard i sind. Die Kette dieser Hashes ist die **Berechnungsspur**; sie macht die Bisektion möglich, ohne Aktivierungen on-chain zu speichern.

6.2 Ganzzahlige Ausführung als Grundlage des Determinismus

Nichtdeterminismus in neuronalen Netzen entsteht nicht aus der Rechengenauigkeit, sondern aus der **Nichtassoziativität der Gleitkommaaddition**: Für Gleitkommazahlen gilt $(a + b) + c \neq a + (b + c)$, weil nach jedem Schritt gerundet wird. Eine Matrixmultiplikation summiert Tausende Produkte, und die Reihenfolge, in der eine GPU Teilergebnisse zusammenführt, hängt von Kernel-Implementierung, Blockaufteilung und Laufzeitbedingungen ab. Zwei ehrliche Knoten erhalten deshalb im Regelfall verschiedene Bits.

Ganzzahladdition ist dagegen assoziativ. Eine Summe ganzer Zahlen liefert dasselbe Ergebnis, gleichgültig in welcher Reihenfolge sie gebildet wird. Führt man die Inferenz vollständig ganzzahlig aus, ist Bitgleichheit deshalb keine Auflage an die Ausführung, sondern eine Eigenschaft der Arithmetik.

Dass eine solche Ausführung möglich ist, ist keine Annahme dieser Arbeit, sondern belegt. Erste Ansätze für Transformer ersetzen die Quadratwurzel in der Normalisierung durch eine L1-Norm-Entsprechung [41], aufbauend auf der dyadischen Arithmetik für Faltungsnetze [42]. I-BERT quantisiert die gesamte Inferenz einschließlich der nichtlinearen Operationen GELU, Softmax und Layer Normalization über ganzzahlige Approximationen und erreicht dabei eine Genauigkeit, die der Gleitkomma-Referenz entspricht oder sie leicht übertrifft [18]. I-ViT bestätigt das für Vision-Transformer [19], und I-LLM überträgt den Ansatz auf große Sprachmodelle [20]. Voraus ging die Beobachtung, dass Transformer-Aktivierungen einzelne Dimensionen mit stark erhöhter Amplitude aufweisen, die bei der Quantisierung gesondert behandelt werden müssen [17]. Der Durchsatz spricht ebenfalls dafür: Gegenüber fp32-Inferenz werden Beschleunigungen um den Faktor 2,4 bis 4 berichtet [18], da Ganzzahleinheiten auf gängiger Hardware breit verfügbar sind.

Verbindlich sind daher nur drei Festlegungen, alle Teil von θ_v :

1. **Vollständig ganzzahlige Ausführung.** Keine Gleitkommaoperation im Inferenzpfad, einschließlich der nichtlinearen Funktionen. Deren Approximationskoeffizienten und Shift-Weiten sind Teil der Modellversion.
2. **Akkumulatorbreite und Bitbreite je Tensor.** Ein 32-Bit-Akkumulator ist vorgeschrieben. Die Bitbreite der Gewichte wird je Tensor festgelegt, nicht global; die Begründung folgt aus Kapitel 6.9. Bei int8-Faktoren beträgt der größte Produktbetrag 16.129; über die üblichen Reduktionslängen bleibt der Abstand zur Überlaufgrenze um mehrere Größenordnungen erhalten (Anhang B.5). Das Überlaufverhalten (Sättigung) ist dennoch explizit festzulegen.

3. Division ausschließlich als arithmetischer Rechtsshift. Dies ist die einzige verbliebene Quelle plattformabhängiger Ergebnisse: Bei negativen Zahlen unterscheiden sich abrundende Division und Trunkierung zur Null, die in verschiedenen Programmiersprachen unterschiedlich implementiert sind. Der arithmetische Rechtsshift ist dagegen auf allen gängigen Architekturen identisch definiert und entspricht durchgängig der Abrundung (Anhang B.5).

Was ausdrücklich nicht vorgeschrieben wird: die Reduktionsreihenfolge, die Blockaufteilung, die Kernel-Implementierung, der Einsatz von Matrixeinheiten. Die Hardware darf parallelisieren, wie sie will. Damit entfällt der Durchsatzverlust, den eine erzwungene Operationsreihenfolge mit sich brächte, und ebenso jede Bevorzugung einzelner Hardware-Klassen. Letzteres ist keine Nebensache: Eine Vorschrift zugunsten hochpräziser Gleitkomma-Akkumulation würde Consumer-Beschleuniger benachteiligen, auf denen dieser Pfad vielfach nur mit halber Rate läuft, während Rechenzentrums-Hardware keinen entsprechenden Malus kennt. Sie liefe damit dem Dezentralisierungsziel des Netzwerks zuwider.

6.3 Verworfen Alternativen

Zwei naheliegende Entwürfe wurden geprüft und verworfen. Die Gründe sind dokumentiert, weil sie für verwandte Arbeiten von Belang sind; die Belege stehen in Anhang B.5.

Gleitkomma mit erzwungener Reihenfolge. Dies ist der Weg von RepOps [15] und vergleichbarer Bibliotheken [16]: Eine feste Reduktionsordnung stellt bitweise Reproduzierbarkeit über verschiedene Hardware her. Der Ansatz ist erprobt, hat für unseren Fall jedoch drei Nachteile. Erstens kostet die eingeschränkte Parallelisierung Durchsatz; berichtet werden je nach Setup erhebliche Aufschläge. Zweitens setzt er voraus, dass die Hardware einem einheitlichen Gleitkommastandard folgt, was für einfache Genauigkeit weitgehend zutrifft, für halbe Genauigkeit und die auf KI ausgelegten Matrixeinheiten jedoch nicht: Deren Rundungsverhalten, Akkumulatorbreiten und Normalisierungspunkte unterscheiden sich zwischen Herstellern und Generationen [21]. Drittens deckt RepOps ausdrücklich nur den Einzelgerätefall ab; Reproduzierbarkeit über mehrere Knoten mit Pipeline- oder Tensor-Parallelismus wird als offene Arbeit benannt [15], und genau dieser Fall liegt in Myelith vor.

Toleranzvergleich statt Bitgleichheit. Statt Gleichheit zu verlangen, ließe sich ein Abstand unterhalb einer Schwelle τ akzeptieren, wie es lokaltätssensitive Commitment-Verfahren für die Prüfung einzelner Anbieter vorschlagen [14]. Drei Befunde sprechen dagegen. Rechenrauschen akkumuliert über verkettete Ausführung so weit, dass die Ergebnisse zweier ehrlicher Knoten nach wenigen Layern so stark auseinanderliegen

wie manipulierte von unmanipulierten. Die erforderliche Trennschärfe ist zudem nicht robust: Unter verletzten Verteilungsannahmen steigt die Anforderung um ein Vielfaches. Und entscheidend: Ein Toleranzband ist adaptiv angreifbar. Wer das Prüfkriterium kennt, richtet die Manipulation daran aus; bei strukturbasierten Commitments genügt es, die geprüften Komponenten korrekt zu berechnen und die übrigen zu verfälschen, was in der Simulation auch über zehn nachfolgende Layer unentdeckt blieb.

Ein dritter Weg wäre, Gleitkommaoperationen vollständig in Software zu emulieren [9]. Er liefert bitgenaue Ergebnisse ohne Anforderungen an die Hardware, ersetzt jedoch jede einzelne Operation durch eine Folge von Ganzzahlbefehlen. Wenn ohnehin ganzzahlig gerechnet wird, ist es folgerichtig, das Modell selbst ganzzahlig auszuführen, statt Gleitkomma darüber nachzubilden.

Ganzzahlige Ausführung vermeidet alle drei Probleme: Sie verlangt keine Reihenfolgenvorschrift, kennt keinen Toleranzbereich, an dem sich ein Angreifer ausrichten könnte, und bildet keine fremde Arithmetik nach.

6.4 Die drei Verifikationsstufen

Stufe 1, Deterministische Redundanz (sofort, günstig).

Jedes Segment wird von $r = 2$ unabhängig zugelosten Pods berechnet. Stimmen die Commitment-Hashes an allen Spur-Positionen überein, gilt das Segment als vorläufig bestätigt. Der Vergleich ist binär und ohne Parameter; es gibt keine Schwelle, die kalibriert, angegriffen oder per Governance verschoben werden könnte.

Zeitpunkt des Abgleichs, zwei Auslieferungsmodi. Der Vergleich kann vor oder nach der Auslieferung erfolgen; beide Varianten sind je Anfrage wählbar:

- **Optimistische Auslieferung (Standard).** Die Antwort des zuerst fertigen Pods wird sofort ausgeliefert, der Abgleich erfolgt asynchron und wirkt über Slashing und Rückbuchung der vTFE-Gutschrift. Die Latenz entspricht der eines einzelnen Pods; die Sicherheit ist nachträglich.
- **Bestätigte Auslieferung (wählbar, Aufpreis).** Die Antwort wird zurückgehalten, bis der Zwillings-Pod übereinstimmt. Ein manipuliertes Ergebnis erreicht den Nutzer nicht, sofern nicht beide Pods kolludieren. Der Preis ist Latenz und ein Gebührenaufschlag.

Die Wahl gehört zum Nutzer: Für eine Recherche genügt nachträgliche Sanktion, für eine Agenten-Entscheidung mit Finanzwirkung (Kap. 8) ist die vorbeugende Variante angemessen.

Stufe 2, Optimistische Stichproben (verzögert, gezielt).

Checker rechnen VRF-ausgeloste Segmente ($\sim 1\text{--}3\%$ des Volumens) nach. Bei Abweichung startet das Bisektions-Spiel (6.6).

Stufe 3, zkML-Anker (selten, teuer, maximal sicher).

Optionaler Premium-Pfad für Ergebnisse mit Finanzwirkung, zugleich Aufrüstpfad, sobald zkML-Systeme effizient genug werden. Ganzzahlige Ausführung kommt diesem Pfad entgegen, da arithmetische Schaltkreise über Ganzzahlen deutlich einfacher zu formulieren sind als über Gleitkomma.

6.5 Ausführungsspezifikation als Protokolleigenschaft

Die Ausführungsspezifikation ist Teil von θ_v und damit konsensrelevant. Sie umfasst das Quantisierungsschema (Bitbreiten für Gewichte und Aktivierungen), die Akkumulatorbreite, das Überlaufverhalten, die Koeffizienten der ganzzahligen Approximationen nichtlinearer Funktionen, die Regeln der dynamischen Quantisierung sowie die Festlegung auf den arithmetischen Rechtsschift.

Ein Miner, der von dieser Spezifikation abweicht (etwa in geringerer Bitbreite rechnet oder Layer überspringt) spart reale Kosten und verschlechtert reale Ausgabequalität. Unter Bitgleichheit ist das kein Grenzfall, sondern unmittelbar sichtbar, da jede Abweichung die Hashes auseinandertreibt. Ebenso gilt: Jede Kompression von Aktivierungen auf der Leitung (Kap. 10) muss protokolldefiniert und für beide redundanten Pods identisch sein, da sie sonst Teil der Ausführungsspezifikation wäre.

Nicht Teil der Spezifikation und damit frei wählbar bleiben Kernel-Implementierung, Parallelisierungsstrategie, Blockgrößen und Speicherlayout. Hier liegt der Freiheitsgrad, der heterogener Hardware Teilnahme ohne Wettbewerbsnachteil erlaubt.

6.6 Das Bisektions-Spiel

Behauptet ein Checker, Segment σ sei falsch, läuft ein interaktives Protokoll mit $O(\log L)$ Runden (L = Anzahl der Shard-Übergänge):

1. Checker legt eigene Spur $h(a'_{0..k})$ vor; erster abweichender Übergang sei j
2. On-chain wird nur Layer-Gruppe j entschieden:
 - Miner j legt $a_{\{j-1\}}$ offen (Erasure-codierte Fragmente aus der DA-Schicht)
 - Validatoren-Komitee führt EINEN Shard-Forward gemäß $\theta_{v,j}$ aus
 - Vergleich mit $h(a_j)$: Miner oder Checker verliert
3. Verlierer: Slash; Gewinner: Kopfgeld

Die Schuldzuweisung ist eindeutig, weil das Ergebnis kanonisch ist: Es gibt genau ein korrektes a_j , und der Vergleich ist eine Hash-Gleichheit ohne Ermessensspielraum. Ganzzahlige Ausführung erleichtert die Schiedsrunde zusätzlich, da die Validatoren keine spezielle Hardware und keine zertifizierte Kernel-Implementierung benötigen; eine Referenzausführung in Ganzzahlarithmetik läuft auf jeder Plattform mit demselben Ergebnis. Die Kosten bleiben konstant: ein einzelner Shard-Forward auf etwa sieben Validatoren, unabhängig von der Segmentlänge und im Regelbetrieb nie fällig.

6.7 Kontrollsegmente

Die drei Stufen sanktionieren Betrug entweder durch Vergleich zweier Berechnungen oder durch nachträgliche Stichprobe. Beide setzen voraus, dass der Zwillings-Pod ehrlich rechnet oder dass ein Angreifer wiederholt auffällt. Für den einmaligen Eingriff eines Angreifers, der beide Pods kontrolliert, bleibt eine Lücke.

Kontrollsegmente (Canaries) verkleinern sie. Das Netzwerk hält einen Vorrat von Segmenten, deren korrektes Ergebnis bereits vorliegt, weil sie unter kontrollierten Bedingungen berechnet wurden. Gateways schleusen solche Segmente mit einem Anteil γ in den regulären Auftragsstrom. Für den ausführenden Miner sind sie von echten Nutzeranfragen nicht unterscheidbar; ihre Prüfung ist trivial, da das Soll-Commitment hinterlegt ist.

Der Sicherheitsgewinn liegt in der **Ungewissheit des Angreifers**: Da er bei keinem Segment weiß, ob es eine Kontrolle ist, trägt bereits der erste Manipulationsversuch ein Entdeckungsrisiko von γ , unabhängig davon, ob er beide Pods kontrolliert. Bei $\gamma = 2\%$ und vollem Stake-Verlust im Entdeckungsfall ist der Erwartungswert eines Einzelangriffs bereits negativ, sofern der Angreifer seinen Stake überhaupt bewertet.

Drei Konstruktionsanforderungen sind wesentlich:

1. **Ununterscheidbarkeit.** Kontrollsegmente müssen der realen Prompt-Verteilung entstammen und im Timing-, Längen- und Kontextprofil unauffällig sein. Erkennt ein Miner sie an statistischen Merkmalen, ist der Mechanismus wirkungslos (Kap. 10, Punkt 5).
2. **Vorratserneuerung.** Ein statischer Pool wird über die Zeit erkennbar. Naheliegend ist die Übernahme abgeschlossener, per Stufe 2 vollständig geprüfter Echtsegmente.
3. **Kostenehrlichkeit.** Kontrollsegmente sind reiner Overhead; γ geht direkt in die Kostenstruktur ein und ist ein Governance-Parameter (Kap. 10.3).

6.8 Sicherheitsargument (Skizze)

Unter den Annahmen (a) $\leq f < 1/3$ byzantinische Validator-Stimmen, (b) mindestens ein ehrlicher Checker prüft jede Stichprobe, (c) die DA-Schicht liefert Aktivierungen während der Streitfrist, gilt:

- **Soundness:** Ein falsches Segment überlebt die Streitfrist nur, wenn beide redundanten Pods identisch falsch rechnen (Kollusion über die VRF-Zulassung hinweg, $P \approx \beta^{2k}$, Anhang B.2), es nicht in der Stichprobe landet ($P = 1-p$) und kein Kontrollsegment trifft ($P = 1-\gamma$). Die Ereignisse sind unabhängig, das Gesamtrisiko multiplikativ. Da der Vergleich binär ist, existiert kein Toleranzbereich, in dem sich eine Manipulation verstecken ließe.
- **Liveness:** Fällt ein Shard-Miner aus, übernimmt der Standby-Miner des Pods ($k+2$ Mitglieder, 2 in Reserve); Session-Verlust nur bei mehr als zwei gleichzeitigen Ausfällen im selben Pod.

6.9 Empirische Prüfung an einer Referenzimplementierung

Die Annahme aus 6.2, dass sich ein Sprachmodell vollständig ganzzahlig ausführen lässt, ohne die Ausgabequalität nennenswert einzubüßen, wurde seit Version 0.2 dieses Papiers an einer Referenzimplementierung geprüft. Grundlage ist ein denses Modell mit 0,5 Milliarden Parametern unter permissiver Lizenz; Gewichte in acht Bit mit kanalweisen Zweierpotenz-Skalen, Aktivierungen in sechzehn Bit mit kalibrierten Skalen je Schicht.

Ergebnis. Die Perplexität auf einem festen Auszug aus WikiText-2 beträgt 15,59 gegenüber 14,95 in der Gleitkomma-Referenz, gemessen mit identischer Methode auf denselben Sequenzen. Der relative Abstand liegt bei 4,3 Prozent. Die Übereinstimmung der jeweils wahrscheinlichsten Token mit der Gleitkomma-Referenz beträgt 89,3 Prozent.

Der Determinismus ist bestätigt. Fünfundzwanzig unabhängige Läufe in fünf Gruppen liefern bitidentische Ergebnisse. Eine Instrumentierung des gesamten Vorwärtspfads findet keine Gleitkommaoperation. Damit ist die Eigenschaft, auf der die Verifikation dieses Kapitels beruht, an einer realen Implementierung nachgewiesen, nicht nur argumentativ begründet.

Wo die Kosten entstehen — und wo nicht. Aufschlussreicher als die Endzahl ist die Zerlegung des Fehlers. Führt man dieselbe Quantisierung in Gleitkomma-Arithmetik aus, ergibt sich:

Konfiguration	Perplexität	Abstand
Gleitkomma, nicht quantisiert	14,95	Referenz
Gewichte in acht Bit, Arithmetik in Gleitkomma	15,05	+0,7 %
Gewichte in sechzehn Bit, Arithmetik in Gleitkomma	14,94	-0,1 %
Vollständig ganzzahliger Pfad	15,59	+4,3 %

Die Gewichtsquantisierung kostet also fast nichts, und eine Verdopplung der Bitbreite bringt keinen Gewinn. Der gesamte Abstand entsteht in den Zwischenstufen des ganzzahligen Pfades: den Quantisierungen zwischen den Teilschritten, den ganzzahligen Approximationen der nichtlinearen Funktionen und der Rundung bei jeder Reskalierung.

Für die Praxis folgt daraus eine klare Priorität: Wer ganzzahlige Ausführung umsetzt, sollte den Aufwand nicht in die Gewichtsdarstellung investieren, sondern in die Genauigkeit der Zwischenberechnungen. Die vorliegende Implementierung erreichte das genannte Ergebnis erst, nachdem mehrere naheliegende Wege — feinere Gewichtsquantisierung nach dem Verfahren von [44], Erhöhung der Gewichtsbitbreite, Verbreiterung des Kalibrierungskorpus — nachweislich ohne Wirkung geblieben waren.

Die Ausführungsspezifikation ist aus den Gewichten zu gewinnen, nicht aus der Architekturbeschreibung. Mehrere Annahmen, die aus der Modellbeschreibung

plausibel abgeleitet waren, erwiesen sich als unzutreffend: das Vorhandensein von Bias-Termen in den Projektionen der Aufmerksamkeit, die Aufteilung der Schlüssel- und Wertköpfe bei gruppierter Abfrage sowie der tatsächlich auftretende Wertebereich am Eingang der Exponentialfunktion. Keiner dieser Punkte betrifft den Determinismus, aber jeder hätte zu falschen Ergebnissen geführt. Für Kapitel 10.1 folgt daraus, dass die Spezifikation ein Messergebnis ist und nicht aus der Dokumentation eines Modells abgeleitet werden kann.

Nicht jeder Tensor verträgt dieselbe Bitbreite. Führt ein Modell Eingabe-Einbettung und Ausgabeprojektion auf demselben Gewicht, unterscheidet sich die Fehlertoleranz beider Verwendungen erheblich. Beim Nachschlagen der Einbettung wirkt der Quantisierungsfehler additiv auf einen Reststrom weit größerer Amplitude und bleibt folgenlos; bei der Ausgabeprojektion entscheidet derselbe Fehler unmittelbar über die Rangfolge der Token. Die Bitbreite ist deshalb je Tensor festzulegen und Teil von θ_v . Der Determinismus bleibt davon unberührt, da eine höhere Bitbreite die Assoziativität nicht verändert; betroffen ist allein der Speicherbedarf.

Grenzen dieses Ergebnisses. Gemessen wurde an einem Modell am unteren Ende der Größenskala, das für Quantisierung den ungünstigsten Fall darstellt: Mit wachsender Modellgröße nehmen die Abstände zwischen den führenden Logits zu, sodass ein gegebener Fehler relativ weniger wiegt. Für die im Papier angestrebte Größenordnung ist damit ein günstigeres Ergebnis zu erwarten, aber nicht belegt. Ebenso steht der Nachweis aus, dass die Bitgleichheit über verschiedene Hardware-Klassen hinweg gilt; die vorliegenden Läufe erfolgten auf einer Referenzimplementierung ohne Beschleuniger (Kap. 11, Punkte 2 und 3).

6.10 Was dieser Entwurf kostet

Der Ehrlichkeit halber die Gegenrechnung. Die ganzzahlige Ausführung vermeidet zwar den Durchsatzverlust einer Reihenfolgenvorschrift, verlangt aber ein **quantisiertes Modell**. Damit bindet sich das Protokoll an eine Modellklasse, deren Qualität gegenüber der Gleitkomma-Referenz nachzuweisen ist. Die vorliegende Literatur ist ermutigend, aber nicht abschließend: Acht-Bit-Quantisierung ist breit belegt [18][19], die Übertragung auf große Sprachmodelle ist jüngeren Datums [20] und für Modelle in der hier angestrebten Größenordnung nicht umfassend validiert. Sollte sich zeigen, dass ganzzahlige Ausführung bei der angestrebten Modellgröße spürbare Qualitätseinbußen verursacht, wäre die Grundlage dieses Kapitels neu zu bewerten (Kap. 10, Punkt 1).

Ferner beruht die Determinismus-Eigenschaft auf der Vollständigkeit der Spezifikation. Übersehene plattformabhängige Operationen (etwa im Verhalten ganzzahliger Matrixeinheiten, bei Sättigung an den Wertebereichsgrenzen oder durch Compiler-Umformungen) könnten Nichtdeterminismus

wieder einführen. Anders als bei Gleitkomma sind solche Fälle jedoch aufzählbar und durch Konformitätstests mit endlich vielen Testvektoren prüfbar; sie erfordern keine Einschränkung der Parallelisierung.

Was auch unter Bitgleichheit bestehen bleibt: Ein Angreifer, der **beide** zugelassenen Pods kontrolliert, kann ein konsistent falsches Ergebnis erzeugen. Dagegen wirken die Zulosung (Kap. 4.3), die erzwungene Zonendiversität der Zwillings-Pods (Kap. 4.4), die Stichprobenprüfung und die Kontrollsegmente, jeweils mit eigener unabhängiger Wahrscheinlichkeit. Ausgeschlossen ist dieser Fall nicht; für Anwendungen, in denen auch das nicht tragbar ist, existiert Stufe 3 mit kryptografischer statt probabilistischer Garantie.

7. Training und Modellentwicklung

Ein Netzwerk, das ein Sprachmodell betreibt, sollte es auch fortschreiben können. Andernfalls veraltet das Modell, während die Kapazität wächst, und das Netzwerk bleibt dauerhaft von externen Trainingsläufen abhängig. Dieses Kapitel beschreibt, wie Training in die bestehende Architektur eingefügt wird, welche Auflagen dafür zwingend sind und wo die Grenzen des Verfahrens liegen.

Die Ausgangslage ist günstiger als erwartet: Die ganzzahlige Ausführung aus Kapitel 6 überträgt sich unverändert auf den Rückwärtspass, da auch die Gradientenberechnung assoziativ ist. Verifizierte Berechnung allein genügt jedoch nicht. Anders als bei Inferenz, wo Korrektheit die gesamte Frage ist, kann beim Training eine bitgleich korrekte Rechnung auf ungeeigneten Daten beruhen oder in schädlicher Richtung wirken. Die Abschnitte 7.3 bis 7.5 behandeln diese Lücke.

7.1 Trainings als nachrangige Arbeitsklasse

Inferenz hat unbedingten Vorrang. Sie erzeugt die Gebühren, aus denen sich das Netzwerk finanziert, und Nutzer erwarten Antwortzeiten, die kein Hintergrundprozess beeinträchtigen darf. Training läuft daher als zweite, nachrangige Arbeitsklasse in der Restkapazität.

Der Scheduler weist Trainingssegmente nur an Pods zu, deren Auslastung in der vorangegangenen Epoche unter einer Schwelle lag, und begrenzt den Anteil auf eine **Grundrate** γ_{train} . Ein Wert im Bereich von fünf bis zehn Prozent der freien Kapazität ist der sinnvolle Ausgangspunkt. Die Bezugsgröße ist dabei zu beachten: γ_{train} bemisst sich an der *freien* Kapazität, nicht an der Gesamtleistung des Netzwerks. Bei einer Auslastung von siebenzig Prozent entsprechen zehn Prozent freier Kapazität etwa drei Prozent der Gesamtleistung und damit der Größenordnung, die die Treasury aus Kapitel 5.3 trägt. Die Obergrenze folgt nicht aus einer festen Zahl, sondern aus der gemessenen Inferenznachfrage: Steigt die Auslastung über das Ziel aus Kapitel 5.4, wird

Training gedrosselt, bevor die Credit-Preise anziehen. Damit ist ausgeschlossen, dass Training Inferenzkapazität verdrängt.

Was diese Kapazität leistet. Bei einem Modell der 24-Milliarden-Klasse und einer Grundrate von zehn Prozent erreicht ein Netz mit 5.000 Minern etwa eine Milliarde Trainings-Token pro Tag, ein Netz mit 50.000 Minern rund neun Milliarden (Anhang B.6). Ein Feintuning-Lauf ist damit in Tagen erreichbar. Ein vollständiges Vortraining, das Billionen Token erfordert, ist es nicht und wird es auch bei erheblichem Wachstum nicht sein. Das Netzwerk kann ein Modell fortschreiben, nicht erzeugen; die Abhängigkeit von einem extern vortrainierten Basismodell ist dauerhaft, nicht anfänglich.

7.2 Lokale Verlustblöcke statt globaler Rückpropagierung

Ganzzahlige Rückpropagierung stößt auf ein Überlaufproblem: Die Fehlerterme wachsen mit jeder rückwärts durchlaufenen Schicht, und bei acht Bit breiten Gewichten überschreiten sie den 32-Bit-Bereich bereits nach wenigen Schichten [22]. Zwei Verfahren lösen das, und beide passen zur vorliegenden Architektur.

Erstens die **Block-Skalierung** aus NITI [23]: Nach jeder Schicht wird der Fehlervektor durch einen gemeinsamen Zweierpotenz-Faktor geteilt, dessen Exponent separat mitgeführt wird. Der Faktor folgt aus dem Betragsmaximum und ist damit reihenfolgeunabhängig; angewandt wird er als arithmetischer Rechtsshift, also mit genau der Operation, die Kapitel 6.2 ohnehin verbindlich festlegt. Über vierzig Schichten tritt damit kein Überlauf auf (Anhang B.6).

Zweitens **lokale Verlustblöcke** [24]: Das Netz wird in Segmente mit eigenen Verlustfunktionen gegliedert, sodass Gradienten das Segment nicht verlassen. Für Myelith ist das mehr als eine Überlaufvermeidung: Legt man die Blockgrenzen auf die Shard-Grenzen, entfällt der Rückwärtspass über die Pipeline vollständig. Es entsteht kein zusätzlicher Netzverkehr, und die Verifikation bleibt lokal, ein Shard-Paar prüft seinen eigenen Gradienten. Der Preis ist eine gegenüber globaler Rückpropagierung schlechtere Lösung [24]; wie groß dieser Abstand bei Sprachmodellen ausfällt, ist offen (Kap. 11, Punkt 3).

7.3 Datenprovenienz statt Datenbewertung

Die schwierigste Frage des Trainings lautet nicht, ob korrekt gerechnet wurde, sondern ob die Daten legitim waren. Ein Miner, der vergiftete Texte einspeist, rechnet bitgleich korrekt und erzeugt dennoch ein verschobenes Modell. Der Bitvergleich aus Kapitel 6 greift hier nicht.

Eine inhaltliche Bewertung der Daten scheidet aus: Sie wäre subjektiv und damit genau jener Bewertungsspielraum, den das Protokoll an anderer Stelle bewusst vermeidet (Kap. 2). Myelith prüft daher

nicht den Inhalt, sondern die **Herkunft**. Das Verfahren ist der Blockchain-gestützten Föderierten-Lernen-Literatur entlehnt [30]–[33] und hier auf den offenen Netzbetrieb übertragen (vgl. Kap. 2).

Das Protokoll führt eine Liste kanonischer Korpora, jedes mit einer Merkle-Wurzel im Konsens verankert. Ein Trainingssegment referenziert keine Rohdaten, sondern einen Merkle-Beweis: Der Textabschnitt steht an einer bestimmten Position im Korpus mit der Wurzel R. Damit wird die Prüfung wieder objektiv und exakt so verifizierbar wie eine Inferenz. Ein Miner kann keine eigenen Daten einschleusen, weil er für nicht existierende Positionen keinen gültigen Beweis erzeugen kann.

Der Aufwand ist gering, sofern Segmente gebündelt zugewiesen werden: Ein einzelner Beweis über einen Korpus von einer Milliarde Dokumenten kostet knapp zwölf Prozent Overhead, ein gemeinsamer Beweis über 256 zusammenhängende Segmente hingegen unter einem halben Prozent (Anhang B.6).

Auswahl bleibt als Angriffsfläche. Wer keine Daten fälschen kann, kann immer noch auswählen. Ein Angreifer mit vierzig Prozent Kapazitätsanteil hätte bei freier Wahl auch vierzig Prozent Einfluss auf die Datenzusammensetzung. Die Datenzuweisung erfolgt deshalb **ebenfalls per VRF**: Welcher Pod welche Korpusabschnitte bearbeitet, ergibt sich aus dem Epochen-Seed, nicht aus der Wahl des Miners. Diesem bleibt nur, zugewiesene Segmente abzulehnen, was Vergütung kostet und über die Ablehnungsquote sichtbar wird. Der Resteinfluss sinkt damit auf wenige Prozent (Anhang B.6). Diese Auflage ist konstitutiv, nicht optional.

7.4 Aggregation und Übernahme

Robuste Aggregation. Die Gradienten vieler Pods müssen zu einem Update zusammengeführt werden. Der naheliegende Mittelwert ist ungeeignet: Ein einzelner extremer Beitrag verschiebt ihn beliebig, und bereits fünf Prozent byzantinische Pods erzeugen eine deutliche Verzerrung (Anhang B.6). Myelith aggregiert daher über den **Median** [35]. Dessen Bruchpunkt liegt bei fünfzig Prozent und fällt damit mit der ohnehin angenommenen byzantinischen Schranke zusammen [35]; getrimmte Mittelwerte versagen dagegen bereits bei einem Drittel Angreiferanteil. Die Verfahren stammen aus der Literatur zu byzantinisch robustem föderiertem Lernen [34][35] und werden hier unverändert übernommen; ihre bekannte Schwäche bei stark ungleich verteilten Daten gilt auch hier. Der Median benötigt nur Vergleiche und bleibt damit deterministisch und im Verifikationsmodell nachprüfbar.

Umgang mit veralteten Gradienten. Pods rechnen unterschiedlich schnell. Jeder Gradient trägt deshalb den Modellstand, auf dem er berechnet wurde; Beiträge, die älter als eine festgelegte Zahl von Schritten sind, werden verworfen. Modellrechnungen zeigen, dass

moderate Verzögerung die Konvergenz bremst, aber nicht verhindert (Anhang B.6); wo die praktische Grenze liegt, ist zu messen.

Übernahme neuer Gewichte. Es gilt der dreistufige Prozess aus Kapitel 10.2 mit zwei Ergänzungen, die aus der Analyse folgen. Erstens wird das **Hold-out-Set der Shadow-Phase erst nach Abschluss des Trainings per VRF aus dem Korpus gezogen** und dann offengelegt. Ein vorab bekannter Benchmark erlaubt andernfalls erheblichen scheinbaren Fortschritt ohne echte Verbesserung. Zweitens umfasst die Bewertung **Regressionstests**: Ein Update, das bestehende Fähigkeiten verschlechtert, wird abgelehnt, auch wenn es neue verbessert.

Wiederholungsanteil gegen Vergessen. Fortlaufendes Training auf neuen Daten lässt bestehende Fähigkeiten verfallen. Ohne Gegenmaßnahme geht ein erheblicher Teil verloren; ein Wiederholungsanteil von etwa fünfzehn Prozent aus dem Bestandskorpus begrenzt den Verlust deutlich (Anhang B.6). Dieser Anteil ist Teil der VRF-gesteuerten Datenzuweisung und damit nicht durch Miner beeinflussbar.

7.5 Modellwachstum

Feintuning erhält ein Modell, vergrößert es aber nicht. Zwischen Feintuning und Vortraining liegt ein dritter Weg, der zu einem wachsenden Netzwerk passt: die schrittweise Vergrößerung des bestehenden Modells.

Funktionserhaltende Expansion. Verfahren wie Net2Net [25] und bert2BERT [26] vergrößern ein Modell, ohne seine Funktion zu verändern: Neuronen werden aufgespalten, neue Schichten als Identität initialisiert. Das vergrößerte Modell verhält sich unmittelbar nach der Expansion identisch zum Vorgänger. Für Myelith folgt daraus zweierlei. Erstens ist ein Wachstumsschritt **ohne Qualitätsrisiko aktivierbar**, da sich das Verhalten zunächst nicht ändert; die Verbesserung entsteht erst durch das anschließende Nachtraining. Zweitens ist die Expansion eine deterministische Transformation der Gewichte und damit **bitgleich verifizierbar** wie jede andere Berechnung. Die neue Modellversion θ_{v+1} ergibt sich reproduzierbar aus θ_v und dem Wachstumsoperator; beide werden on-chain verankert.

Kosten. Ein Wachstumsschritt erfordert etwa ein Drittel der Token, die ein Vortraining derselben Größe kosten würde, da das Vorwissen erhalten bleibt (Anhang B.6). Die Literatur berichtet für progressives Wachstum Einsparungen zwischen vierzig und knapp siebzig Prozent [27][28][29].

Strukturelle Kopplung. Tiefenwachstum fügt Schichten hinzu, in der Pipeline also **zusätzliche Shards**. Mehr Miner ermöglichen mehr Shards, mehr Shards tragen mehr Schichten. Netz- und Modellwachstum sind damit nicht nur zeitlich, sondern architektonisch verbunden. Ein erwünschter Nebeneffekt: Die Kollusionsschranke β^{2k} aus Anhang B.2 verbessert sich mit steigendem k, Wachstum erhöht also auch die Sicherheit.

Zeitskala, nüchtern betrachtet. Ein Netz mit 500 Minern kann nicht wachsen, der erste Schritt läge jenseits von sieben Jahren. Mit 5.000 Minern dauert er etwa neun Monate, mit 50.000 Minern rund einen Monat, spätere Schritte entsprechend länger (Anhang B.6). Wachstum ist damit kein kontinuierlicher Prozess, sondern ein seltenes Ereignis im Jahresmaßstab, das an erhebliche Netzgröße gebunden ist. Der Zeitpunkt wird zur Governance-Entscheidung: Zu frühes Wachstum verschlechtert die Qualität je Parameter, zu spätes verschrenkt Kapazität.

7.6 Was dieser Entwurf offenlässt

Drei Punkte bleiben ungelöst und werden hier benannt statt umschrieben.

Die Finanzierung erzeugt in jeder Variante Fehlanreize. Training verbrennt keine Credits und erzeugt damit keinen Burn, aus dem sich eine Vergütung ableiten ließe. Eine Vergütung nach Rechenzeit belohnt Training unabhängig davon, ob es dem Modell nützt, und schafft damit dieselbe Fehlanreizstruktur, die das Protokoll bei der Inferenz vermeidet. Eine ergebnisabhängige Vergütung wäre dagegen subjektiv und angreifbar. Der vorliegende Entwurf finanziert Training aus der Protokoll-Treasury (Kap. 5.3), ergänzt um einen per Governance abschaltbaren Aufschlag auf Inferenzgebühren, und verzichtet auf einen Anteil an der Prägung. Das ist ein Kompromiss, keine Lösung: Es begrenzt die Fehlanreize durch ein Budget, statt sie zu beseitigen.

Die Kombination der Verfahren ist unbelegt. Ganzzahliges Training ist belegt [23][24], Modellwachstum ist belegt [25]–[29], die Kombination beider ist es nicht. Zudem stammen die Belege für ganzzahliges Training aus dem Bildbereich mit vergleichsweise kleinen Netzen; für Transformer in der hier angestrebten Größenordnung liegt kein Nachweis vor.

Das Verhalten unter offenen Netzbedingungen ist unbekannt. Sämtliche Literatur zu progressivem Wachstum entstammt zentral kontrollierten Läufen mit einheitlicher Hardware, ununterbrochenem Zeitplan und freier Datenwahl. Ob dieselben Verfahren unter heterogener Kapazität, unterbrochenen Läufen und VRF-zugewiesenen Daten funktionieren, ist offen.

8. Agent Layer

Der Agent Layer macht aus dem Inferenznetz ein handlungsfähiges System: Ein Agent plant über mehrere Schritte, ruft Werkzeuge auf und kann Transaktionen auslösen. Damit verlässt er den Bereich, in dem das Verifikationsmodell aus Kapitel 6 trägt, denn dieses beruht auf der Reproduzierbarkeit von Berechnungen. Die Außenwelt ist nicht reproduzierbar. Dieses Kapitel beschreibt, wo die Grenze verläuft, wie sie sichtbar gemacht wird und wie der Schaden jenseits davon begrenzt bleibt.

8.1 Die Grenze der Verifizierbarkeit

Stufe 1 vergleicht die Ergebnisse zweier Pods auf Bitgleichheit. Ruft ein Agent eine Websuche oder eine externe Schnittstelle auf, erhalten beide Pods verschiedene Antworten, da sie zu verschiedenen Zeitpunkten anfragen und die Daten sich ändern. Der Vergleich schlägt dann fehl, ohne dass ein Fehler vorliegt.

Das Protokoll löst dies, indem Werkzeugergebnisse aus der Berechnung herausgenommen und zur **Eingabe** gemacht werden: Ein Gateway ruft das Ergebnis einmal ab, versieht es mit Zeitstempel und Signatur und übergibt es beiden Pods als identischen Text. Die Signatur geht in die Berechnungsspur ein und wird damit mitverifiziert. Was verifiziert wird, ist die *Verarbeitung* der Antwort, nicht ihre Richtigkeit.

Daraus folgt eine Unterscheidung, die dem Nutzer sichtbar gemacht wird:

- **Deterministische Werkzeuge** liefern reproduzierbare Antworten: Abfragen des eigenen Ledgers, Berechnungen, Zugriffe auf im Konsens verankerte Korpora (Kap. 7.3). Sie werden vollständig verifiziert wie jede andere Berechnung.
- **Externe Werkzeuge** liefern nicht reproduzierbare Antworten: Websuche, Marktdaten, fremde Schnittstellen. Ihre Antwort ist attestiert, aber nicht verifiziert; das Protokoll bezeugt, dass ein bestimmtes Gateway zu einem bestimmten Zeitpunkt diese Antwort erhalten hat, nicht dass sie zutrifft.

Für externe Werkzeuge besteht damit ein Vertrauensanker beim abrufenden Gateway. Mehrfachabruf durch unabhängige Gateways mildert das, wo die Antwort stabil ist, versagt jedoch bei sich laufend ändernden Daten. Diese Einschränkung wird benannt, nicht verschleiert.

8.2 Session-Kontrakte und Schadensbegrenzung

Ein Agent, der Transaktionen auslösen kann, verwandelt einen Berechnungsfehler in einen Vermögensschaden. Der Restfall aus Kapitel 6.10, ein manipuliertes Segment, das alle Prüfungen übersteht, wird hier erst wirklich teuer. Die Antwort des Protokolls ist nicht, den Fall auszuschließen, sondern seine Auswirkung zu begrenzen.

Jede Agenten-Session läuft unter einem **Session-Kontrakt** mit vier durchgesetzten Grenzen:

1. **Gesamtbudget** in Credits und, sofern Transaktionen erlaubt sind, in MYL.
2. **Einzeltransaktionslimit**, unabhängig vom Restbudget.
3. **Empfänger-Whitelist**: Adressen, an die überhaupt gezahlt werden darf.
4. **Zeitfenster**, nach dessen Ablauf die Session erlischt.

Entscheidend ist, wo diese Parameter stehen: **im Kontrakt, nicht im Kontext des Modells**. Sie sind für den Agenten nicht lesbar und nicht änderbar. Kein Text, den der Agent verarbeitet, kann sie beeinflussen; die Durchsetzung erfolgt beim Ausführen der Transaktion durch den Konsens.

Hinzu tritt eine **Kopplung von Betragshöhe und Sicherheitsstufe**: Transaktionen oberhalb eines im Kontrakt festgelegten Schwellenwerts werden nur ausgeführt, wenn das zugrundeliegende Segment im Modus bestätigter Auslieferung berechnet wurde (Kap. 6.4), bei dem beide redundanten Pods übereinstimmen mussten, bevor das Ergebnis den Nutzer erreichte. Wer höhere Beträge zulässt, zahlt dafür mit Latenz und Gebühr.

8.3 Umgang mit eingeschleusten Anweisungen

Verarbeitet ein Agent fremde Inhalte, können diese Anweisungen enthalten, die sich als Nutzauftrag ausgeben. Dieses Problem ist bekannt und ungelöst; filterbasierte Ansätze gelten als unzuverlässig, da der Prüfmechanismus derselben Angriffsfläche unterliegt wie das Modell.

Myelith folgt daher dem Ansatz architektonischer Trennung, wie ihn das Dual-LLM-Muster [39] und dessen Ausarbeitung in CaMeL [40] beschreiben: Der planende Teil sieht keine fremden Inhalte, der verarbeitende Teil kann keine Werkzeuge aufrufen, und abgerufene Daten können den Kontrollfluss nicht beeinflussen. Für das vorliegende Protokoll ergibt sich daraus eine natürliche Verstärkung: Die Berechtigungen liegen ohnehin im Session-Kontrakt und damit außerhalb der Reichweite des Modells (8.2). Ein eingeschleuster Text kann den Agenten täuschen, aber weder sein Budget erhöhen noch einen Empfänger hinzufügen.

Damit verschiebt sich das Problem von der Sicherheit zur Ergebnisqualität: Ein getäuschter Agent kann eine schlechte Entscheidung innerhalb seiner Grenzen treffen, aber nicht darüber hinaus handeln. Das ist die stärkste verfügbare Aussage; eine vollständige Abwehr wird von den zitierten Arbeiten ausdrücklich nicht beansprucht [40].

8.4 Verkettung der Schritte

Ein Agent arbeitet iterativ: schlussfolgern, Werkzeug aufrufen, weiter schlussfolgern. Jeder Schritt ist ein eigenes Inferenz-Segment mit eigener Verifikation. Damit auch der *Ablauf* nachprüfbar bleibt, referenziert jedes Segment das Ausgabe-Commitment seines Vorgängers. Es entsteht eine Kette, die dieselbe Struktur hat wie die Berechnungsspur innerhalb eines Segments (Kap. 6.1), nur eine Ebene höher.

Prüfbar ist damit nicht nur, ob jeder Schritt korrekt gerechnet wurde, sondern auch, dass keine Schritte ausgelassen, eingefügt oder vertauscht wurden. Abbruchbedingungen (Höchstzahl der Schritte, Budgeterschöpfung, Zielerreichung) stehen im Session-Kontrakt und werden vom Konsens durchgesetzt, nicht vom Agenten selbst.

Der persistente Speicher einer Session, etwa ein Vektorspeicher für das Agentengedächtnis, wird als eigene Arbeitsklasse innerhalb des Netzwerks betrieben und unterliegt derselben Provenienzanforderung wie Trainingsdaten (Kap. 7.3): Was in den Speicher eingeht, muss auf ein verifiziertes Segment zurückführbar sein.

8.5 Verantwortung

Für den Fall, dass ein Agent Schaden anrichtet, hält das Protokoll keine technische Lösung bereit, und es wird auch keine behauptet. Was es leistet, ist vollständige Nachvollziehbarkeit: Aus der Segmentkette und den Attestierungen lässt sich rekonstruieren, welcher Pod welchen Schritt berechnet hat, welche Werkzeugantworten eingingen und welches Gateway sie attestierte.

Was es nicht leistet: eine Zusicherung, dass der Agent richtig entscheidet. Es kann der Fall eintreten, dass jeder Beteiligte korrekt gehandelt hat, das Protokoll fehlerfrei arbeitete und dennoch ein Schaden entstand, weil das Modell eine schlechte Entscheidung traf oder eine externe Antwort falsch war. Wer einem Agenten Verfügungsrechte einräumt, trägt dieses Risiko.

Sinngemäß gelten die Vertraulichkeitsklassen aus Kapitel 9.3 auch hier: Agenten mit Transaktionsrechten sind für Vorgänge geeignet, deren möglicher Schaden das gesetzte Budget nicht übersteigt, und ungeeignet, wo eine Fehlentscheidung nicht durch ein Budget begrenzt werden kann.

9. Vertraulichkeit und Risikomodell

9.1 Wo die Vertraulichkeitsgrenze verläuft

Myelith ist ein offenes Netzwerk fremder Rechner. Die entscheidende Eigenschaft, die Nutzer kennen müssen, lautet: **Shard-Miner sehen die Aktivierungen der von ihnen berechneten Segmente.** Aktivierungen sind keine belanglosen Zwischenwerte. Aus ihnen lässt sich der Eingabetext in erheblichem Umfang rekonstruieren. Wer ein Segment rechnet, kann also im Prinzip erfahren, worum es darin geht.

Diese Eigenschaft ist keine Implementierungslücke, sondern folgt zwingend daraus, dass fremde Hardware das Modell ausführt. Sie lässt sich nicht wegverschlüsseln, solange auf Klartext gerechnet wird.

9.2 Was Verschlüsselung an den Rändern leistet, und was nicht

Alle Verbindungen zwischen Nutzer, Gateway und den Endpunkten der Pipeline (erster und letzter Shard) sind Ende-zu-Ende verschlüsselt; ebenso die Aktivierungs-Streams zwischen den Shards. Das schließt eine reale und nicht kleine Angreiferklasse aus: Netzbeobachter, Zwischenknoten, kompromittierte Gateways sowie Miner, die nicht selbst am betreffenden Segment beteiligt sind. Da Gateways nur weiterleiten und nicht rechnen, entfällt damit ein Sammelpunkt, an dem sonst der gesamte Klartext-Verkehr vieler Nutzer zusammenliefe.

Was Verschlüsselung ausdrücklich nicht leistet: Schutz vor den beteiligten Shard-Minern selbst. Ihre Aufgabe ist die Verarbeitung des Inhalts.

9.3 Risikoklassen für Nutzer

Anstelle unpräziser Zusicherungen benennt das Protokoll ausdrücklich, wofür es geeignet ist:

Klasse	Beispiele	Eignung
A. Öffentlich	Öffentliche Dokumente, Recherche, Code aus offenen Repositorien, kreative Arbeit	Geeignet. Kein Vertraulichkeitsverlust, da ohnehin öffentlich.
B. Intern, geringe Sensitivität	Interne Notizen, unkritische Geschäftsvorgänge	Bedingt geeignet. Die Pod-Zusammensetzung wechselt je Epoche; kein einzelner Miner sieht mehr als Ausschnitte.
C. Vertraulich	Personenbezogene Daten, Gesundheits- und Finanzdaten, Geschäftsgeheimnisse, Rechtsberatung	Ungeeignet. Nicht verwenden, solange keine hardwaregestützte Vertraulichkeit besteht.

Die Segmentierung dämpft das Risiko der Klasse B zusätzlich: Eine Session wird in Segmente zerlegt, deren Zuteilung sich je Epoche ändert, sodass ein einzelner Miner typischerweise nur Bruchstücke sieht. Das ist eine Erschwernis, keine Garantie: Mehrere kollidierende Miner können Bruchstücke zusammenführen.

9.4 Pfad zu höherer Vertraulichkeit

Zwei Wege sind absehbar, beide mit Nachteilen, die transparent benannt gehören:

- **Vertrauenswürdige Ausführungsumgebungen (TEEs)** auf Miner-Seite würden Klasse C ermöglichen, führen aber Vertrauen in Chiphersteller ein. Ein Zentralisierungsfaktor, der dem Grundgedanken des Netzwerks widerspricht, und ein Angriffsziel mit dokumentierter Bruchgeschichte. Denkbar als *optionale* Kapazitätsklasse mit eigener Vergütung, deren Nutzung Nutzer bewusst wählen, nicht als Netzwerkstandard.
- **Homomorphe Verschlüsselung** würde das Problem grundsätzlich lösen, ist für Modelle dieser Größenordnung jedoch um Größenordnungen zu langsam. Beobachtungsgegenstand, keine Planungsgrundlage.

Bis dahin gilt die Klassifikation aus 8.3 unverändert. Ein System, das seine Grenzen klar benennt, ist einem vorzuziehen, das Vertraulichkeit suggeriert, die es nicht leisten kann.

10. Governance und Modell-Herkunft

10.1 Das Modell als Allmende

Da die Gewichte zwangsläufig auf Miner-Hardware liegen, muss das Netzwerkmodell open-weight sein. Der Genesis-Zustand referenziert ein existierendes Modell über die Merkle-Wurzel seiner quantisierten Gewichte.

Anforderungen an das Basismodell. Aus der Architektur folgen vier Kriterien, die die Auswahl stärker einschränken als die bloße Verfügbarkeit:

1. **Permissive Lizenz.** Erforderlich sind Apache 2.0 oder MIT. Lizenzen mit Nutzerzahl-Obergrenzen oder geografischen Beschränkungen scheiden aus, da ein offenes Protokoll seine Nutzerzahl weder kennt noch begrenzen kann. Apache 2.0 ist gegenüber MIT vorzuziehen, da es einen ausdrücklichen Patentgrant enthält.
2. **Dense statt Mixture-of-Experts.** MoE-Modelle leiten jeden Token dynamisch zu wechselnden Experten, sodass der Datenpfad je Token variiert. Die feste Pod-Kette aus Kapitel 4 setzt einen konstanten Pfad voraus. Dense-Modelle sind je Parameter teurer, aber architektonisch verträglich.
3. **Moderate Schichtzahl bei hoher Qualität je Parameter.** Jeder Shard-Übergang kostet Netzlatenz. Modelle, die ihre Qualität eher über Breite als über Tiefe erreichen, sind für WAN-Pipelines im Vorteil.
4. **Ganzzahlige Quantisierbarkeit.** Das Modell muss vollständig ganzzahlig ausführbar sein, einschließlich der nichtlinearen Operationen (Kap. 6.2).

Stand der Verfügbarkeit. Die dritte und vierte Anforderung sind erfüllbar, aber nicht fertig verfügbar.

Für diese Modelle der 24-Milliarden-Klasse unter Apache 2.0 existieren INT8-Quantisierungen, die Speicherbedarf etwa halbieren und den Matrixdurchsatz etwa verdoppeln. Diese quantisieren jedoch nur die linearen Operatoren innerhalb der Transformer-Blöcke; Softmax, Layer Normalization und die Aktivierungsfunktionen verbleiben in Gleitkomma. Für Myelith genügt das nicht: Verbleibt auch nur eine Gleitkommaoperation im Pfad, entfällt die Determinismus-Eigenschaft aus Kapitel 6.2.

Die Erstellung einer vollständig ganzzahligen Quantisierung nach dem Vorbild von [18] und [20] ist damit eine notwendige Vorarbeit vor dem Genesis-Block und zugleich der erste Prüfstein für die Tragfähigkeit des Entwurfs (Kap. 11, Punkt 1).

Wer verantwortet die Quantisierung. Das quantisierte Modell ist Teil von θ_v und damit konsensrelevant. Seine Erstellung muss deshalb reproduzierbar dokumentiert sein: Ausgangsgewichte, Quantisierungsverfahren, Kalibrierungsdaten und Werkzeugversionen gehen in das Genesis-Manifest ein, sodass jeder Teilnehmer die Ableitung nachvollziehen kann. Andernfalls entstünde an dieser Stelle ein Vertrauensanker, den das Protokoll sonst überall vermeidet.

10.2 Modell-Updates

Updates (neue Versionen θ_{v+1}) durchlaufen einen dreistufigen Prozess:

1. **Vorschlag:** Treasury-finanzierte oder externe Teams reichen Kandidaten-Gewichte mit reproduzierbarem Trainings-/Finetuning-Protokoll ein.
2. **Shadow-Phase:** 5 % der Pod-Kapazität betreibt den Kandidaten parallel; öffentliche Benchmark-Suite läuft on-chain-attestiert.
3. **Abstimmung:** Stimmgewicht = Stake $\times$ Arbeitshistorie (wie Validatoren-Wahl). Bei Annahme: koordinierter Gewichts-Rollout über eine Übergangsepoche mit doppelter Versionshaltung.

Langfristig kann eine zweite Arbeitsklasse (Trainings-Segmente, verifiziert à la Gensyn) das Finetuning selbst ins Netzwerk holen; für v1 ist das bewusst ausgeklammert (Komplexität).

10.3 Parameter-Governance

Per Abstimmung änderbar: Stichprobenrate p , Subventionsrate s , Kernel-Whitelist, Auslastungsziel, Streitfrist. Nicht änderbar (Verfassungsrang): Gesamtangebot, Burn-and-Mint-Prinzip, Determinismus-Pflicht der Runtime.

11. Offene Forschungsfragen

Die folgenden Punkte sind bewusst als *Messfragen* formuliert: Jeder benennt die Größe, die zu bestimmen ist, und den Meilenstein, in dem das geschieht.

1. **Ausgabequalität ganzzahliger Inferenz in der Zielgrößenordnung (M0).** *Gemessen für 0,5 Milliarden Parameter (Kap. 6.9): 4,3 Prozent Abstand zur Gleitkomma-Referenz.* Offen bleibt die Übertragung auf die angestrebte Größenordnung, für die ein günstigeres Ergebnis zu erwarten, aber nicht belegt ist. Die Verifikation aus Kapitel 6 setzt ein vollständig ganzzahlig ausführbares Modell voraus. Acht-Bit-Quantisierung ist für Transformer breit belegt [18][19], die Übertragung auf große Sprachmodelle ist jüngeren Datums [20] und für die angestrebte Größenordnung nicht umfassend validiert. Zu messen ist der Qualitätsabstand zur Gleitkomma-Referenz auf etablierten Benchmarks. Fällt er zu groß aus, ist die Grundlage des Kapitels neu zu bewerten; die Rückfalloption wären toleranzbasierte Commitments (Punkt 10).
2. **Vollständigkeit der Ausführungsspezifikation (M0).** *Teilergebnis liegt vor (Kap. 6.9):* Die Spezifikation ist aus den Gewichten zu gewinnen, nicht aus der Dokumentation; auf einer Referenzimplementierung ohne Beschleuniger ist sie vollständig. Offen bleibt der Nachweis über verschiedene Hardware-Klassen hinweg. Die Determinismus-Eigenschaft beruht darauf, dass alle plattformabhängigen Operationen erfasst sind. Zu prüfen sind insbesondere das Verhalten ganzzahliger Matrixeinheiten an den Bereichsgrenzen, die Sättigungssemantik, mögliche Compiler-Umformungen sowie die Approximationen der nichtlinearen Funktionen. Ergebnis ist eine Konformitätssuite mit Testvektoren, die neue Hardware ohne Protokolländerung aufnimmt.
3. **Durchsatz auf heterogener Hardware (M0/M1).** Für ganzzahlige Inferenz werden gegenüber fp32 Beschleunigungen um den Faktor 2,4 bis 4 berichtet [18][19]. Zu messen ist, ob dieser Vorteil über NVIDIA-, AMD-, Apple- und CPU-Hardware hinweg gleichmäßig auftritt; ungleiche Verteilung wäre ein Zentralisierungsrisiko.
4. **Rasterweite der Token-Auswahl (M1).** Die Auswahl erfolgt deterministisch aus quantisierten Logits. Zu messen sind der Qualitätseinfluss der Quantisierung und die Häufigkeit von Grenzfällen exakt auf Rasterlinien.
5. **Ununterscheidbarkeit von Kontrollsegmenten (M2).** Der Sicherheitsgewinn aus 6.7 steht und fällt damit, dass Miner Canaries nicht erkennen. Zu untersuchen ist, woran sie sich statistisch identifizieren ließen (Prompt-Verteilung, Länge, Kontextaufbau, Wiederkehr, Timing) und ob die Übernahme geprüfter Echtsegmente diese Merkmale beseitigt. Zu bestimmen ist ferner der Anteil γ als Abwägung zwischen Abschreckung und Overhead.

6. Pod-Latenz gegen lokale Kollusionsdichte (M1). Die Regel aus Kapitel 4.4 ist qualitativ begründet, aber unquantifiziert. Zu messen: erreichbare Pipeline-Latenz je Zonengröße gegen die resultierende lokale Kapazitätskonzentration β_{lokal} .

7. Aktivierungskompression (M1). Bandbreite ist der Engpass des WAN-Betriebs. Ganzzahlige Aktivierungen kommen der Kompression entgegen, da sie bereits quantisiert vorliegen. Zu prüfen ist, ob Delta-Kodierung zwischen aufeinanderfolgenden Token die Übertragungsmenge deutlich senkt. Bedingung: Jede Kompression muss protokolldefiniert und für beide redundanten Pods identisch sein (Kap. 6.5).

8. Verteilter Prefix-Cache (M1/M2). Geteilte Präfixe versprechen erheblichen Durchsatzgewinn. Vor einer Aufnahme sind zwei Sicherheitsfragen zu klären: der Timing-Seitenkanal, über den ein Angreifer die Existenz fremder Präfixe erkennen könnte, sowie die Bedingung, dass nur Stufe-1-bestätigte Präfixe cachefähig sein dürfen.

9. Ganzzahliges Training bei Sprachmodellen (M3). Die Belege für ganzzahliges Training stammen aus dem Bildbereich mit vergleichsweise kleinen Netzen [23][24]. Zu messen ist der Qualitätsabstand zu Gleitkomma-Training bei Transformern in der Zielgrößenordnung sowie der Abstand lokaler Verlustblöcke zu globaler Rückpropagierung (Kap. 7.2).

10. Kombination von ganzzahligem Training und Modellwachstum (M3/M4). Beide Verfahren sind einzeln belegt, ihre Kombination nicht. Zu prüfen ist insbesondere, ob funktionserhaltende Expansion unter ganzzahliger Darstellung exakt bleibt oder Rundungsfehler einführt (Kap. 7.5).

11. Progressives Wachstum unter offenen Netzbedingungen (M4). Sämtliche Literatur entstammt zentral kontrollierten Läufen. Zu untersuchen ist das Verhalten bei heterogener Kapazität, unterbrochenen Läufen und VRF-zugewiesenen Daten.

12. Finanzierung des Trainings. Der Entwurf finanziert Training aus Treasury und optionalem Gebührenaufschlag und benennt in Kapitel 7.6, dass jede Variante Fehlanreize erzeugt. Zu suchen ist ein Mechanismus, der Trainingsbeiträge nach Nutzen vergütet, ohne subjektive Bewertung einzuführen. Dies ist die schwächste Stelle des Entwurfs.

13. Attestierung externer Werkzeuge (M4). Für nicht reproduzierbare Werkzeugantworten besteht ein Vertrauensanker beim abrufenden Gateway (Kap. 8.1). Zu untersuchen ist, für welche Antwortklassen Mehrfachabruf durch unabhängige Gateways tragfähig ist und ab welcher Änderungsrate er versagt.

14. Restwirksamkeit eingeschleuster Anweisungen (M4). Die architektonische Trennung [39][40] verhindert Grenzüberschreitungen, nicht aber

Fehlentscheidungen innerhalb der Grenzen. Zu messen ist, welcher Schaden innerhalb eines gegebenen Budgets durch Täuschung tatsächlich erreichbar ist.

15. Modell-Gewichte als Allmende. Unverändert offen: Die Gewichte liegen zwangsläufig bei den Minern, das Modell muss open-weight sein. Woher kommt das Basismodell? Hinzu kommt die Frage, wer die ganzzahlige Quantisierung durchführt und verantwortet, da das quantisierte Modell Teil von θ_v ist (Kap. 10.2).

16. Verifikation ohne Determinismus-Anforderung. Anhang B.5 begründet, warum die geprüften Toleranzverfahren nicht tragen, insbesondere wegen adaptiver Angreifbarkeit. Ein Verfahren, das gegen einen Angreifer robust ist, der das Prüfkriterium kennt, würde die Bindung an quantisierte Modelle aufheben. Dies ist die aussichtsreichste Richtung für eine künftige Fassung.

17. Vertraulichkeit jenseits Klasse B. Klasse C bleibt außer Reichweite, solange auf Klartext gerechnet wird (Kap. 9). TEE-Kapazität als optionale Klasse ist entwerfbar, führt aber Herstellervertrauen ein; homomorphe Verfahren bleiben Beobachtungsgegenstand.

18. Isomorphe Rollenverteilung als Alternative zur Redundanz. VeriLLM lässt Inferenz- und Prüfrollen auf denselben Knoten laufen und vermeidet so einen getrennten Prüferbestand [43]. Zu untersuchen ist, ob sich dieses Muster mit der Zulosung aus Kapitel 4.3 verträgt, ohne die Unabhängigkeitsannahme zu verletzen, und ob es den Redundanz-Overhead unter die 50 Prozent aus Punkt 19 senken könnte.

19. Ökonomie des Redundanzfaktors. $r = 2$ halbiert die Effizienz gegenüber zentralen Anbietern. Zu prüfen bleibt, ob adaptive Redundanz ($r = 1$ bei Minern mit langer sauberer Historie) die Kosten senkt, ohne die Unabhängigkeitsannahme zu verletzen.

Anhang A: Kern-Datentypen und Referenzalgorithmen

Dieser Anhang dokumentiert die protokollrelevanten Datentypen und die drei Kernalgorithmen der Referenzimplementierung. Engineering-Dokumentation (Repository-Struktur, Build-Konventionen, Implementierungs-Meilensteine, CI) findet sich im Projekt-Repository unter `docs/`.

A.1 Kern-Datentypen (`myl-types`)

```
pub struct Segment {
  pub id: SegmentId,           // h(session || index)
  pub input_commitment: Hash,   // h(prompt_chunk || kv_root)
  pub model_version: MerkleRoot, // Gewichts-Wurzel 0_v inkl. Ausführungsspezifikation
  pub pod_path: Vec<MinerId>,   // Pipeline-Reihenfolge
  pub output_commitment: Hash,
  pub trace: Vec<ActivationHash>, // h(a_0), ..., h(a_k): Berechnungsspur
  pub signatures: Vec<BlsSignature>, // eine pro Shard-Übergang
}

pub struct PoIBundle {
  pub epoch: EpochId,
  pub pod: PodId,
  pub segments_root: MerkleRoot, // über alle Segment-Ids der Epoche
  pub vtfc_claimed: u64,         // beanspruchte Arbeit
  pub aggregate_sig: BlsSignature, // aggregiert über Pod-Mitglieder
}

pub struct InferenceCredit { pub owner: Address, pub vtfc: u64, pub expiry: EpochId }
```

A.2 Epochen-Scheduler (myl-scheduler), deterministische Zuteilung

```
/// Von JEDEM Node identisch nachrechenbar, kein zentraler Scheduler.
pub fn assign_epoch(
  seed: VrfOutput,           // aus finalisiertem Block der Vorepoche
  miners: &[MinerRegistration], // Registrierungsschluss: Epoche e-2
  latency_graph: &LatencyGraph, // gossip-attestierten Paarlatenzen
  cfg: &ShardConfig,         // k Shards, Pod-Größe k+2
) -> EpochAssignment {
  // 1. Miner nach Hardware-Klasse filtern (VRAM ≥ Shard-Größe)
  // 2. Geo-Clustering unter Latenz-Constraint:
  //   Pods so bilden, dass max. Paarlatenz im Pod < L_max (z. B. 80 ms),
  //   Clusterwahl aber seed-randomisiert (Kollusionsschutz, Kap. 9 Punkt 2)
  // 3. Shard-Zuweisung INNERHALB des Pods: Fisher-Yates mit seed
  // 4. Redundanz: jedes Nachfrage-Bucket → 2 disjunkte Pods
  // 5. Stichproben-Lotterie: p_i | segments | Segmente für Checker markieren
}
```

A.3 Pipeline-Algorithmus (myl-pod), der "Mining-Loop"

Der Mining-Loop ist bei uPol kein Hash-Raten, sondern der Inferenz-Serviceloop:

```
/// Hauptschleife eines Shard-Miners (Shard i im Pod)
async fn shard_loop(shard: ShardWeights, role: PodRole) {
  loop {
    // 1. Aktivierungen vom Vorgänger empfangen (oder Prompt-Embedding, wenn i == 0)
    let (a_prev, seg) = recv_activations().await;

    // 2. Eingangs-Hash gegen Spur prüfen & deterministischen Kontext setzen
    verify_hash(&a_prev, seg.trace[i - 1])?;
    let ctx = DeterministicCtx::new(seg.id, seg.sampling_seed());

    // 3. Forward-Pass gemäß theta_v (vorgeschriebene Reduktionsreihenfolge)
    let a_next = shard.forward_deterministic(&a_prev, &ctx);

    // 4. Spur fortschreiben, signieren, weiterreichen
    let h_next = hash(&a_next);
    sign_transition(seg.id, seg.trace[i - 1], h_next);
    send_activations(next_peer(), a_next, seg).await;

    // 5. KV-Cache der Session lokal fortschreiben (Session-Affinität)
    kv_cache.update(seg.session_id, &a_next);

    // 6. Aktivierungen erasure-coded für Streitfrist archivieren (DA-Pflicht)
    da_store.put(seg.id, i, encode_fragments(&a_prev));
  }
}

/// Pod-Koordinator: Micro-Batching + PoI-Aggregation
async fn coordinator_loop() {
  loop {
    let batch = intake.collect_microbatch(WINDOW_MS).await; // Pipelining
    dispatch_pipeline(batch).await;
    if epoch_boundary() {
      let bundle = build_poi_bundle(completed_segments());
      submit_to_consensus(bundle).await; // → myl-poi
    }
  }
}
```

A.4 Verifikations-Protokoll (myl-verifier)

```
/// Checker: Stichprobe nachrechnen; Vergleich auf Bitgleichheit.
async fn audit(seg: Segment) -> Option<Challenge> {
  let my_trace = rerun(&seg).await; // eigener Durchlauf, gleiche Reihenfolge
  let j = first_divergence(&seg.trace, &my_trace)?; // None → alles korrekt
  Some(Challenge { seg_id: seg.id, layer_group: j, bond: CHECKER_BOND,
    claimed_hash: my_trace[j] })
}

/// On-chain-Schiedsrunde (Validatoren-Komitee): genau EIN Shard-Forward.
/// Das Ergebnis ist kanonisch: es gibt genau ein korrektes a_j.
fn adjudicate(ch: Challenge, fragments: DaFragments) -> Verdict {
  let a_in = decode(fragments)?; // a_{j-1} aus DA-Schicht
  let a_out = runtime::forward(ch.layer_group, &a_in); // gemäß theta_v
  if hash(&a_out) == ch.claimed_hash { Verdict::SlashMiner }
  else { Verdict::SlashChecker }
}
```

A.5 Konsens-Anbindung (myl-consensus + myl-ledger)

```
Block ::= { txs, poi_bundles, challenges, verdicts, epoch_meta }
```

```
State-Übergänge (myl-ledger):
burn(addr, syn) → mint_credits(addr, syn / preis_e)
settle_epoch(e) → M_e berechnen (EMA-Burn), Rewards verteilen
apply_verdict(v) → Stake slashen, Kopfgeld auszahlen, vTFE rückbuchten
credit_spend(session, vtfc) → Session-Budget abbuchen (Agent-Kontrakt)
```

Anhang B: Anreiz-Herleitung

B.1 Sicherheitsbedingung für Shard-Miner

Ein rationaler Miner erwägt, Segmente falsch zu berechnen (z. B. Rechenarbeit einzusparen und Zufallswerte zu liefern). Modell:

- g = Gewinn pro betrogenem Segment (ersparte Rechenkosten $\approx$ Segment-Reward),
- p = Wahrscheinlichkeit, dass ein Segment per Stichprobe geprüft wird,
- s = Stake des Miners, bei Überführung vollständig gelasht,
- Miner betrügt bei Anteil q seiner Segmente.

Redundanz (Stufe 1) erkennt Betrug bereits, wenn der redundante Pod ehrlich ist. Betrug lohnt also überhaupt nur bei Kollusion beider Pods oder wenn der Miner darauf spekuliert, dass Abweichungen als Fehler ohne Slash gewertet würden. Konservativ betrachten wir den ungünstigsten Fall, dass nur die Stichprobe (Stufe 2) Betrug sanktioniert.

Erwarteter Gewinn pro Epoche mit n Segmenten: $E[G] = q \cdot n \cdot g$

Entdeckungswahrscheinlichkeit pro Epoche: $P_d = 1 - (1-p)^{q \cdot n} \approx q \cdot n \cdot p$ (für kleine $p \cdot q \cdot n$)

Erwartete Strafe: $E[S] = P_d \cdot s \approx q \cdot n \cdot p \cdot s$

Ehrlichkeit ist dominant, wenn $E[S] > E[G]$ für alle $q > 0$:

$$qnpS > qng \iff S > g/p$$

Die im Haupttext genannte schärfere Schranke $S_{\min} = g/p^2$ ergibt sich, wenn man zusätzlich verlangt, dass sich Betrug auch über den Zeithorizont bis zur ersten erwarteten Prüfung ($\approx 1/p$ Segmente) nicht amortisiert, also gegen Miner, die nach kurzem Betrugsfenster mit Exit rechnen (Hit-and-Run):

$$S_{\min} = g/p \cdot 1/p = g/p^2$$

wobei g/p dem Gewinn bis zur erwarteten Entdeckung entspricht.

Zahlenbeispiel: $p = 0,02$, Segment-Reward $g = 0,5$ MYL folgt $S_{\min} = 1250$ MYL pro Segment-Kapazität. Ein Miner mit Kapazität von 100 Segmenten/Epoche benötigt Stake in Höhe von ~ 1250 Segment-Rewards $\approx 12,5$ Epochen-Einkommen. Bei Stunden-Epochen: rund ein halber Tag Einkommen als Pfand pro Kapazitätseinheit. Praktikabel, und per p -Erhöhung verschärfbar, falls Betrugsfälle beobachtet werden.

B.2 Kollusionswahrscheinlichkeit der Redundanz

Sei β der Anteil kolludierender Miner-Kapazität. Beide redundanten Pods (je k Shard-Positionen) müssen vollständig kolludieren, um ein falsches Segment durch Stufe 1 zu bringen:

$$P_{\text{koll}} \approx \beta^{2k}$$

Bei $\beta = 0,2$ und $k = 8$: $P_{\text{koll}} \approx 2,6 \cdot 10^{-12}$. Selbst bei $\beta = 0,5$: $1,5 \cdot 10^{-5}$, und jedes solche Segment trägt weiterhin das Stichprobenrisiko p mit Voll-Slash *aller* $2k$ beteiligten Stakes. Die geografische Clusterung der Pod-Bildung (Kap. 4.3) erhöht β lokal; eine für Meilenstein M1 geplante Analyse (Anhang B.9) soll quantifizieren, wie viel Seed-Zufall der Clusterwahl beigemischt werden muss, um β_{lokal} unter einer Zielschranke zu halten (Kap. 9 Punkt 2).

B.3 Checker-Anreize

Checker-Vergütung = Grundvergütung (4 % der Prägung, proportional zu geprüftem Volumen) + Kopfgeld $b \cdot s$ aus Slashes ($b = 30$ %). Die Grundvergütung stellt sicher, dass Prüfen auch bei Betrugsrate ≈ 0 rentabel bleibt, denn das System darf nicht davon abhängen, dass Betrug existiert. Falsche Anfechtungen kosten die Kautio; die Kautio ist so bemessen, dass Spam-Anfechtungen (Erzwingen teurer Schiedsrunden) unrentabel sind: Kautio > Kosten der On-chain-Schiedsrunde $\times$ Sicherheitsfaktor.

B.4 Self-Dealing (Formalisierung von 5.6)

Ein Angreifer mit Kapazitätsanteil α verbrennt X MYL, um Prägung zu ernten. Rückfluss: $\alpha \cdot M_e$. Im Gleichgewicht ($M_e \approx B_e$, EMA-gedämpft) gilt für den Grenzertrag des zusätzlichen Burns ΔX :

$$\alpha \cdot \Delta X \cdot w < \Delta X \text{ für alle } \alpha < 1/w$$

mit dem EMA-Gewicht $w < 1$ und dem Kapazitätsanteil α des Angreifers.

Da $w \approx 1/30$ (EMA-Fenster) und $\alpha \leq 1$, ist Self-Dealing im Gleichgewicht strikt verlustbringend.

Subventionsphase ($s > 0$). Verschärfte Bedingung: Die Modellrechnung zeigt, dass in der Bootstrap-Phase der reine Burn-Mint-Vergleich nicht genügt: Mit Prägung $M_e = B_e \cdot (1+s)$ erntet ein Self-Dealer nominell mehr, als er verbrennt. Die Sicherheit beruht hier auf der Arbeitsbindung der Prägung. Rewards fließen nur gegen verifizierte Rechenarbeit, deren reale Kosten (Hardware, Strom; Anteil c am Reward, empirisch $c \approx 0,6-0,8$) der Angreifer wie jeder Miner trägt. Self-Dealing ist verlustbringend genau dann, wenn

$$s < c/(1 - c)$$

Bei $c = 0,7$ also $s < 2,33$, die Start-Subvention $s = 0,5$ liegt weit darunter. Diese Ungleichung ist als **Governance-Invariante** zu führen: s darf nie in die Nähe von $c/(1-c)$ angehoben werden.

B.5 Ganzzahlige Ausführung und die verworfenen Alternativen

Die Verifikation aus Kapitel 6 beruht auf einer arithmetischen Eigenschaft und zwei Zusatzfestlegungen. Beides ist nachstehend belegt; die zugehörigen Programme sind in B.9 verzeichnet.

B.5.1 Assoziativität als Grundlage. Simuliert man eine Reduktion über 8.192 Terme, wie sie einer Matrixzeile entspricht, und vergleicht vier Reihenfolgen (sequentiell, paarweiser Baum, Split-K mit acht Teilsummen, zufällige Reihenfolge), so liefert die ganzzahlige Rechnung in allen 200 Durchläufen identische Ergebnisse. Dieselbe Rechnung in einfacher Gleitkommagenauigkeit stimmt nur in 9 von 200 Fällen über alle Reihenfolgen hinweg überein; in 96 Prozent der Fälle divergieren die Ergebnisse. Der Determinismus ganzzahliger Ausführung ist damit keine Auflage an die Implementierung, sondern eine Eigenschaft der Operation.

B.5.2 Überlaufreserve. Bei int8-Faktoren beträgt der größte Produktbetrag $127 \cdot 127 = 16.129$. Ein 16-Bit-Akkumulator trägt damit nur zwei Terme und scheidet aus. Ein 32-Bit-Akkumulator trägt rechnerisch über 133.000 Terme; die empirisch größte Summe über 8.192 Terme lag bei rund 1,3 Millionen, was einer Reserve um den Faktor 1.639 entspricht. Ein 32-Bit-Akkumulator ist somit ausreichend dimensioniert. Das Verhalten an der Bereichsgrenze (Sättigung) ist gleichwohl festzulegen, damit es nicht implementierungsabhängig bleibt.

B.5.3 Nichtlineare Operationen und dynamische Quantisierung. Eine ganzzahlige Softmax-Approximation nach dem Vorbild von [18][20] lieferte unter drei verschiedenen Summationsreihenfolgen in 100 von 100 Fällen identische Ergebnisse. Ebenso erwies sich die dynamische Quantisierung als reihenfolgeunabhängig: Der aus dem Betragsmaximum abgeleitete Skalierungsfaktor stimmte in 200 von 200 Fällen überein, da Maximumbildung und Ganzzahldivision nicht von der Elementreihenfolge abhängen. Auch die Teile der Inferenz, die über die reine Matrixmultiplikation hinausgehen, bleiben damit deterministisch.

B.5.4 Die einzige verbliebene Fallgrube: Division bei negativen Zahlen. Abrundende Division, Trunkierung zur Null und arithmetischer Rechtsshift stimmen für negative Operanden nicht überein: -7 geteilt durch 2 ergibt je nach Konvention -4 oder -3. In drei von fünf geprüften Fällen divergierten die Verfahren. Da Programmiersprachen hier unterschiedlich verfahren, wäre dies eine reale Quelle plattformabhängiger Ergebnisse. Die Festlegung auf den arithmetischen Rechtsshift löst das Problem vollständig: Über 100.000 Zufallsfälle stimmte der Shift ausnahmslos mit der abrundenden Division überein, und er ist auf allen gängigen Architekturen als Instruktion identisch definiert. Anders als bei einer Reihenfolgenvorschrift kostet diese Festlegung keinen Durchsatz und ist mit

endlich vielen Testvektoren vollständig prüfbar.

B.5.5 Warum Gleitkomma mit fester Reihenfolge verworfen wurde. Der Ansatz ist erprobt [15], trägt für den vorliegenden Fall jedoch nicht. Er schränkt die Parallelisierung ein und kostet damit Durchsatz; er setzt einheitliches Rundungsverhalten der einzelnen Instruktion voraus, was für die auf KI ausgelegten Matrixeinheiten nicht gegeben ist [21]; und er ist bislang nur für den Einzelgerätefall ausgearbeitet, während Reproduzierbarkeit über mehrere Knoten mit Pipeline-Parallelismus als offene Arbeit benannt wird [15]. Hinzu kommt ein Zentralisierungseffekt: Eine Vorschrift zugunsten hochpräziser Akkumulation benachteiligt Consumer-Beschleuniger, auf denen dieser Pfad vielfach nur mit halber Rate läuft, während Rechenzentrums-Hardware keinen entsprechenden Malus kennt.

B.5.6 Warum ein Toleranzmodell verworfen wurde. Ein Abstandsvergleich unterhalb einer Schwelle τ [14] wurde in vier Punkten geprüft. Erstens verlangt ein zulässiges τ selbst unter günstigen Annahmen, dass Manipulationen mindestens das Fünffache des legitimen Rauschens erzeugen; unter verletzten Verteilungsannahmen steigt die Anforderung auf das Zwanzig- bis Fünfunddreißigfache. Zweitens akkumuliert Rauschen über verkettete Ausführung so stark, dass die Ergebnisse zweier ehrlicher Knoten nach wenigen Layern so weit auseinanderliegen wie manipulierte von unmanipulierten. Drittens erkennt ein strukturbasiertes Kriterium Präzisionsmanipulationen nicht, da Quantisierung die Rangfolge dominanter Komponenten kaum verändert. Viertens, und entscheidend: Ein Toleranzband ist adaptiv angreifbar. Wer das Kriterium kennt, berechnet die geprüften Komponenten korrekt und verfälscht die übrigen; in der Simulation blieb ein so konstruierter Angriff auch über zehn nachfolgende Layer unentdeckt, während er einen erheblichen Teil der Rechenarbeit einspartete.

B.5.7 Grenzen dieser Analysen. Es handelt sich um Modellrechnungen in Software, nicht um Messungen auf Beschleunigern. Belegt ist die arithmetische Eigenschaft, nicht die Bitgleichheit einer vollständigen Transformer-Inferenz auf realer Hardware. Offen bleiben insbesondere das Verhalten ganzzahliger Matrixeinheiten an den Bereichsgrenzen, mögliche Compiler-Umformungen und die Frage, ob die Ausführungsspezifikation vollständig ist. Anders als bei Gleitkomma sind diese Fälle jedoch aufzählbar und durch Konformitätstests prüfbar (Kap. 10, Punkt 2).

B.6 Belege zum Training

Die Aussagen aus Kapitel 7 beruhen auf Modellrechnungen, deren Programme in B.9 verzeichnet sind.

B.6.1 Determinismus des Rückwärtspasses. Die Gradientenberechnung ist eine Summe von Produkten aus Aktivierung und Fehlerterm und damit ebenso

assoziativ wie der Vorwärtspass. Über 200 Durchläufe mit drei Summationsreihenfolgen ergaben sich ausnahmslos identische Ergebnisse. Das Verifikationsmodell aus Kapitel 6 überträgt sich damit unverändert.

B.6.2 Überlauf und Block-Skalierung. Ohne Gegenmaßnahme überschreiten die Fehlerterme bei acht Bit breiten Gewichten und einem 32-Bit-Akkumulator den Wertebereich bereits nach zwei Rückwärtsschritten und wachsen danach exponentiell bis auf 78 Bit. Mit der Block-Skalierung nach [23] bleibt der Fehlervektor über vierzig Schichten stabil bei etwa fünfzehn Bit; kein Überlauf tritt auf. Da der Skalierungsexponent aus dem Betragsmaximum folgt und als arithmetischer Rechtsshift angewandt wird, bleibt die Operation reihenfolgeunabhängig.

B.6.3 Trainingskapazität. Bei einem Modell der 24-Milliarden-Klasse, einem Aufwand von $6 \cdot N$ FLOPs je Token und Redundanzfaktor 2 ergibt sich bei zehn Prozent Grundrate: 500 Miner erreichen rund 98 Millionen Token täglich, 5.000 Miner etwa eine Milliarde, 50.000 Miner rund neun Milliarden. Ein Feintuning-Lauf von einer Milliarde Token ist damit ab mittlerer Netzgröße in einem Tag erreichbar. Ein Vortraining, das Billionen Token erfordert, bleibt um Größenordnungen außer Reichweite.

B.6.4 Kosten der Datenprovenienz. Bei einem Korpus von einer Milliarde Dokumenten beträgt die Merkle-Tiefe 30, ein Einzelbeweis also 960 Byte gegenüber 8.192 Byte Nutzdaten je Segment, mithin 11,7 Prozent Overhead. Werden zusammenhängende Segmente gebündelt zugewiesen, teilen sie den gemeinsamen Teilbaum: Bei 16 Segmenten sinkt der Overhead auf ein Prozent, bei 256 auf 0,42 Prozent.

B.6.5 Auswahl-Poisoning. Bei freier Datenwahl entspricht der Einfluss eines Angreifers seinem Kapazitätsanteil: 40 Prozent Anteil ergeben 40 Prozent Einfluss auf die Datenzusammensetzung. Erfolgt die Zuweisung per VRF, bleibt nur die Ablehnung zugewiesener Segmente; der Resteinfluss sinkt auf etwa zwei Prozent und wird zudem über die Ablehnungsquote sichtbar.

B.6.6 Robuste Aggregation. Simuliert wurde die Abweichung des aggregierten Gradienten vom ehrlichen Wert bei byzantinischen Beiträgen. Der Mittelwert weicht bereits bei fünf Prozent Angreiferanteil um 0,76 ab, bei zwanzig Prozent um 3,80. Der Median bleibt bei denselben Anteilen bei 0,03 beziehungsweise 0,10 und hält auch bei einem Drittel Angreiferanteil (0,19). Der getrimmte Mittelwert versagt dort erwartungsgemäß (3,91), da die Trimmung von zwanzig Prozent nicht ausreicht. Daraus folgt die Wahl des Medians, dessen Bruchpunkt bei fünfzig Prozent liegt.

B.6.7 Veraltete Gradienten. Auf einer konvexen Zielfunktion konvergiert asynchrones Verfahren auch bei bis zu fünfzig Schritten Verzögerung. Diese Aussage ist eingeschränkt: Das Modell ist quadratisch und damit erheblich gutmütiger als die Verlustlandschaft eines

Sprachmodells. Belegt ist damit nur, dass Verzögerung kein prinzipielles Hindernis darstellt; die praktische Grenze ist zu messen.

B.6.8 Benchmark-Manipulation. Ein vorab bekanntes Testset erlaubt in der Modellrechnung etwa 35 Prozent scheinbaren Fortschritt ohne echte Verbesserung. Wird das Hold-out-Set erst nach Abschluss des Trainings per VRF aus dem Korpus gezogen, ist eine Optimierung darauf ausgeschlossen, da die Auswahl weder vorhersagbar noch nachträglich beeinflussbar ist.

B.6.9 Katastrophales Vergessen. Ohne Wiederholungsdaten fällt die Leistung auf bestehenden Fähigkeiten in der Modellrechnung auf etwa vierzig Prozent. Bereits fünf Prozent Wiederholungsanteil heben sie auf sechzig Prozent, fünfzehn Prozent auf zweiundachtzig Prozent, bei entsprechend geringerem Zugewinn auf den neuen Daten. Der Wiederholungsanteil ist damit ein Abwägungsparameter, kein Optimum.

B.6.10 Modellwachstum. Ein Wachstumsschritt von 24 auf 32 Milliarden Parameter erfordert etwa 200 Milliarden Token gegenüber 640 Milliarden für ein Vortraining derselben Größe, ein Verhältnis von rund 1 zu 3,2; spätere Schritte liegen bei etwa 1 zu 3,0. Bei zehn Prozent Grundrate dauert der erste Schritt mit 500 Minern über sieben Jahre, mit 5.000 Minern etwa 263 Tage, mit 50.000 Minern rund dreißig Tage. Die Tiefe wächst dabei etwa mit der Wurzel der Parameterzahl, sodass ein Modell von 24 auf 100 Milliarden Parameter von rund 48 auf 98 Schichten und damit von fünf auf zehn Shards wächst.

B.6.11 Grenzen dieser Analysen. Sämtliche Werte stammen aus Modellrechnungen in Software, nicht aus Trainingsläufen. Die Kapazitätsrechnung beruht auf einer Effizienzannahme von 25 Prozent der Spitzenleistung über WAN; die Wachstumsrechnung auf einer konservativ mit fünfzig Prozent angesetzten Einsparung gegenüber Vortraining. Die Modelle für Vergessen und Benchmark-Manipulation sind qualitativ und dienen der Größenordnung, nicht der Vorhersage.

B.7 Training und Burn-and-Mint-Kreislauf

Training erzeugt Arbeit, aber keinen Burn. Geprüft wurde, ob das den Kreislauf aus Kapitel 5 stört.

B.7.1 Wirkung auf die Netto-Inflation. Über 2.000 Epochen ergibt der reine Inferenzbetrieb eine Netto-Inflation von 11,8 Prozent, getragen von der auslaufenden Subvention. Wird Training aus zusätzlicher Prägung finanziert, steigt sie auf 23,0 Prozent, verdoppelt sich also nahezu. Finanzierung aus der Treasury lässt sie unverändert, da lediglich vorhandene Prägung umverteilt wird; ein Gebührenaufschlag senkt sie geringfügig auf 11,76 Prozent, da er den Burn im selben Maß erhöht wie die Ausgabe. Daraus folgt die Festlegung in Kapitel 5.3.

B.7.2 Bezugsgrößen. Die Grundrate aus Kapitel 7.1 bemisst sich an der freien Kapazität, der Treasury-Anteil

aus Kapitel 5.3 an der Prägung. Bei siebzig Prozent Auslastung entsprechen zehn Prozent freier Kapazität rund drei Prozent der Gesamtleistung. Beide Größen sind damit verträglich; die Treasury deckt Trainingsanteile bis etwa drei Prozent der Prägung vollständig.

B.7.3 Verdrängung der Inferenz. Miner wählen zwischen beiden Arbeitsklassen nach der Vergütung je Rechenstunde. Liegt die Trainingsvergütung darunter, wird Training nur bei freier Kapazität ausgeführt, wie beabsichtigt. Bei Gleichstand entscheidet allein die Zuteilung; liegt sie darüber, verdrängt Training die Inferenz und damit die Einnahmequelle des Netzwerks. Die Obergrenze aus Kapitel 5.3 ist deshalb keine Feinjustierung, sondern eine Stabilitätsbedingung.

B.7.4 Rückkopplung über die Nachfrage. Simuliert wurde der Fall, dass Training die Modellqualität verbessert und dadurch die Nachfrage steigt. Bereits eine schwache Wirkung erhöht das kumulierte Burn-Volumen messbar, eine deutliche Wirkung erheblich. Training finanziert sich damit langfristig über den Kreislauf selbst, sofern die Qualitätsverbesserung tatsächlich eintritt. Bleibt sie aus, ist die Ausgabe ein reiner Verlust, der aus der Treasury getragen wird und dort sichtbar bleibt.

B.7.5 Grenzen. Wie die übrigen Rechnungen in diesem Anhang handelt es sich um ein Modell, nicht um eine Messung. Insbesondere die Rückkopplung zwischen Modellqualität und Nachfrage ist qualitativ angesetzt; ihre tatsächliche Stärke ist die entscheidende offene Größe für die Wirtschaftlichkeit des Trainings (Kap. 11, Punkt 12).

B.8 Ausgabestruktur

Belege zu Kapitel 5.7.

B.8.1 Bedarf der Anlaufphase. Bei der Zielrate von zwei Prozent beträgt $S_{\min}$ je Kapazitätseinheit 1.250 MYL. Für fünfzig Startminer ergibt das einen Stake-Bedarf von 62.500 MYL, dem ein Credit-Bedarf der ersten Nutzer von etwa 540 MYL gegenübersteht. Der Stake bestimmt somit allein die Größenordnung der Anfangsmenge.

B.8.2 Wirkung der Stichprobenrate. Da $S_{\min}$ quadratisch von p abhängt, sinkt der Bedarf für zweihundert Miner von 250.000 MYL bei zwei Prozent auf 40.000 bei fünf, 10.000 bei zehn, 1.600 bei fünfundzwanzig und 400 MYL bei fünfzig Prozent. Eine erhöhte Prüfrate in der Anlaufphase reduziert die erforderliche Anfangsmenge damit um mehr als zwei Größenordnungen.

B.8.3 Wirkung eines Emissionsdeckels. Simuliert wurden zehn Jahre bei wachsender Nachfrage. Ohne Deckel steigt der Umlauf von 100.000 auf rund 31 Millionen MYL, wobei die Prägung dem geglätteten Burn folgt. Ein Deckel oberhalb der anfänglichen Prägung wirkt zunächst nicht, bindet aber mit wachsender Nachfrage und lässt den Umlauf nach zehn Jahren auf 150.000 MYL zurückfallen. Ein von Beginn an bindender Deckel hält den Umlauf dauerhaft bei etwa 100.000 MYL. In beiden Fällen wird mehr verbrannt als geprägt, geleistete Arbeit

also nicht mehr vollständig vergütet. Ein Deckel wirkt damit nicht als Knappheitsgarantie, sondern als Kapazitätsbremse.

B.8.4 Frühphasen-Konzentration. Bei einer Halbierung der jährlichen Prägung entfallen rund 28 Prozent der Fünfjahresemission auf das erste Jahr, unabhängig davon, ob das Netz auf 500, 5.000 oder 50.000 Miner wächst. Der Vorteil früher Teilnahme hängt damit nicht vom späteren Wachstum ab, sondern allein vom Verlauf der Subventionskurve.

B.8.5 Grenzen. Die Nachfrageentwicklung ist als stetiges Wachstum mit logarithmisch normalverteilter Schwankung modelliert. Sprunghafte Nachfrageänderungen, Netzwerkabspaltungen und externe Preiseffekte sind nicht abgebildet.

B.9 Verzeichnis der Simulationen

Die Modellrechnungen dieses Anhangs liegen als ausführbare Programme im Projekt-Repository unter `py/simulations/` bei. Sie benötigen keine Abhängigkeiten über die Standardbibliothek hinaus.

Programm	Gegenstand	Belegt in
<code>tokenomics_sim.py</code>	Burn-and-Mint-Gleichgewicht, Self-Dealing über die Subventionsphase	B.4
<code>tau_sim.py</code>	Erforderliche Trennschärfe eines Toleranzverfahrens	B.5.1
<code>robustness_sim.py</code>	Empfindlichkeit dieser Trennschärfe gegenüber verletzten Verteilungsannahmen	B.5.2
<code>hardware_noise_sim.py</code>	Rauschpegel gängiger Beschleunigerklassen aus Spezifikationsdaten	B.5.3
<code>accum_alternatives_sim.py</code>	Zwischenlösungen bei der Akkumulationsgenauigkeit	B.5.3
<code>topk_stability_sim.py</code>	Strukturbasierte Commitments und der adaptive Angriff	B.5.4
<code>integer_determinism_sim.py</code>	Assoziativität, Überlaufreserve, Divisionssemantik	B.5.1, B.5.4
<code>integer_training_sim.py</code>	Determinismus des Rückwärtspasses, Block-Skalierung	B.6.1, B.6.2
<code>training_capacity_sim.py</code>	Trainingsdurchsatz, Kosten der Datenprovenienz, Auswahl-Poisoning	B.6.3 bis B.6.5
<code>training_integrity_sim.py</code>	Robuste Aggregation, veraltete Gradienten, Benchmark-Manipulation, Vergessen	B.6.6 bis B.6.9
<code>model_growth_sim.py</code>	Kosten und Zeitskala von Wachstumsschritten	B.6.10
<code>training_tokenomics_sim.py</code>	Wechselwirkung von Training und Burn-and-Mint-Kreislauf	B.7
<code>genesis_supply_sim.py</code>	Anlaufphase, Emissionsverlauf, Frühphasen-Konzentration	B.8
<code>latency_sim.py</code>	Pod-Latenz gegen lokale Kollusionsdichte (geplant, Meilenstein Mi)	B.2

1. Chen et al.: HadAgent: Harness-Aware Decentralized Agentic AI Serving with Proof-of-Inference Blockchain Consensus. arXiv:2604.18614, 2026. <https://doi.org/10.48550/arXiv.2604.18614>
2. Qubic Project: Useful Proof of Work / Aigarth. Projektdokumentation. <https://docs.qubic.org/>
3. Rao et al.: Bittensor: A Peer-to-Peer Intelligence Market. Whitepaper, 2021. <https://www.bittensor.com/whitepaper>
4. Borzunov et al.: Petals: Collaborative Inference and Fine-tuning of Large Models. arXiv:2209.01188, 2022. <https://doi.org/10.48550/arXiv.2209.01188>
5. Gensyn: Litepaper: Verifiable Deep Learning Compute Protocol. Technischer Bericht. <https://www.gensyn.ai/>
6. Conway et al.: opML: Optimistic Machine Learning on Blockchain. arXiv:2401.17555, 2024. <https://doi.org/10.48550/arXiv.2401.17555>
7. Design and Evaluation of Cost-Aware Proof of Quality for Decentralized LLM Inference. arXiv:2512.16317, 2025. <https://doi.org/10.48550/arXiv.2512.16317>
8. PolyLink: A Blockchain-Based Decentralized Edge AI Platform for LLM Inference. arXiv:2510.02395, 2025. <https://doi.org/10.48550/arXiv.2510.02395>
9. DeServe: Towards Affordable Offline LLM Inference via Decentralization. arXiv:2501.14784, 2025. <https://doi.org/10.48550/arXiv.2501.14784>
10. Teutsch, Reitwießner: A Scalable Verification Solution for Blockchains (Truebit). Whitepaper, 2017. <https://truebit.io/>
11. Kalodner et al.: Arbitrum: Scalable, Private Smart Contracts. USENIX Security, 2018. <https://doi.org/10.5555/3277203.3277233>
12. Yin et al.: HotStuff: BFT Consensus in the Lens of Blockchain. PODC, 2019. <https://doi.org/10.1145/3293611.3331591>
13. Nakamoto: Bitcoin: A Peer-to-Peer Electronic Cash System. Whitepaper, 2008. <https://bitcoin.org/bitcoin.pdf>
14. Ong et al.: TOPLOC: A Locality-Sensitive Hashing Scheme for Trustless Verifiable Inference. arXiv:2501.16007, 2025. <https://doi.org/10.48550/arXiv.2501.16007>
15. Arun et al.: Verde: Verification via Refereed Delegation for Machine Learning Programs. arXiv:2502.19405, 2025. <https://doi.org/10.48550/arXiv.2502.19405>
16. Microsoft Research: RepDL: Reproducible Deep Learning. Softwarebibliothek. <https://github.com/microsoft/RepDL>
17. Dettmers et al.: LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. arXiv:2208.07339, 2022. <https://doi.org/10.48550/arXiv.2208.07339>
18. Kim et al.: I-BERT: Integer-only BERT Quantization. ICML, 2021. arXiv:2101.01321. <https://doi.org/10.48550/arXiv.2101.01321>
19. Li, Gu: I-ViT: Integer-only Quantization for Efficient Vision Transformer Inference. arXiv:2207.01405, 2022. <https://doi.org/10.48550/arXiv.2207.01405>
20. Hu et al.: I-LLM: Efficient Integer-Only Inference for Fully-Quantized Low-Bit Large Language Models. arXiv:2405.17849, 2024. <https://doi.org/10.48550/arXiv.2405.17849>
21. Accurate Models of NVIDIA Tensor Cores. arXiv:2512.07004, 2026. <https://doi.org/10.48550/arXiv.2512.07004>
22. Song et al.: PocketNN: Integer-only Training and Inference of Neural Networks via Direct Feedback Alignment. arXiv:2201.02863, 2022. <https://doi.org/10.48550/arXiv.2201.02863>
23. Wang et al.: NITI: Training Integer Neural Networks Using Integer-only Arithmetic. IEEE TPDS, 2022. arXiv:2009.13108. <https://doi.org/10.48550/arXiv.2009.13108>
24. Pirillo et al.: NITRO-D: Native Integer-only Training of Deep Convolutional Neural Networks. arXiv:2407.11698, 2024. <https://doi.org/10.48550/arXiv.2407.11698>
25. Chen et al.: Net2Net: Accelerating Learning via Knowledge Transfer. arXiv:1511.05641, 2015. <https://doi.org/10.48550/arXiv.1511.05641>
26. Chen et al.: bert2BERT: Towards Reusable Pretrained Language Models. ACL, 2022. <https://aclanthology.org/2022.acl-long.151/>

27. Gong et al.: Efficient Training of BERT by Progressively Stacking. ICML, 2019. arXiv:1906.02926. <https://doi.org/10.48550/arXiv.1906.02926>
28. Wang et al.: Learning to Grow Pretrained Models for Efficient Transformer Training (LiGO). ICLR, 2023. arXiv:2303.00980. <https://doi.org/10.48550/arXiv.2303.00980>
29. Du et al.: Stacking Your Transformers: A Closer Look at Model Growth for Efficient LLM Pre-Training. NeurIPS, 2024. <https://papers.nips.cc/>
30. Blockchain-enabled Data Integrity for Federated Learning: Merkle-based Provenance and Auditable Update Trails. Discover Artificial Intelligence, 2026. <https://link.springer.com/journal/44163>
31. Yang et al.: TrustDFL: A Blockchain-Based Verifiable and Trusty Decentralized Federated Learning Framework. Electronics 13(1), 2024. <https://doi.org/10.3390/electronics13010123>
32. PoCQ: Proof of Contribution Quality as a Lightweight Blockchain Consensus for Secure Federated Learning. arXiv:2606.05642, 2026. <https://doi.org/10.48550/arXiv.2606.05642>
33. FIDELIS: Blockchain-Enabled Protection Against Poisoning Attacks in Federated Learning. arXiv:2508.10042, 2025. <https://doi.org/10.48550/arXiv.2508.10042>
34. Blanchard et al.: Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent (Krum). NeurIPS, 2017. <https://doi.org/10.5555/3295222.3295384>
35. Yin et al.: Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates. ICML, 2018. <https://doi.org/10.5555/3295222.3295382>
36. Bentov et al.: Cryptocurrencies without Proof of Work. arXiv:1406.5694, 2014. <https://doi.org/10.48550/arXiv.1406.5694>
37. KRNC: New Foundations for Permissionless Byzantine Consensus and Global Monetary Stability. arXiv:1909.07433, 2019. <https://doi.org/10.48550/arXiv.1909.07433>
38. Fair Launch Token Distribution: Mechanisms, Benefits and Limitations. Übersichtsdarstellung, 2025.
39. Willison: The Dual LLM Pattern for Building AI Assistants That Can Resist Prompt Injection. Blogbeitrag, 2023. <https://simonwillison.net/2023/Apr/25/dual-llm-pattern/>
40. DeBenedetti et al.: Defeating Prompt Injections by Design (CaMeL). arXiv:2503.18813, 2025. <https://doi.org/10.48550/arXiv.2503.18813>
41. Lin et al.: Towards Fully 8-bit Integer Inference for the Transformer Model. IJCAI, 2020. arXiv:2009.08034. <https://doi.org/10.48550/arXiv.2009.08034>
42. Jacob et al.: Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. CVPR, 2018. arXiv:1712.05877. <https://doi.org/10.48550/arXiv.1712.05877>
43. VeriLLM: A Lightweight Framework for Publicly Verifiable Decentralized Inference. arXiv:2509.24257, 2026. <https://doi.org/10.48550/arXiv.2509.24257>
44. Frantar et al.: GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. ICLR, 2023. arXiv:2210.17323. <https://doi.org/10.48550/arXiv.2210.17323>

Hinweis zur Redlichkeit

Referenz [1] nimmt Begriff und Grundidee des Proof of Inference vorweg; die Referenzen [14] und [15] enthalten die beiden Verifikationsbausteine, die das Modell in Kapitel 6 kombiniert. Die Abgrenzung der eigenen Beiträge ist in Kapitel 2 dargelegt. Wo eine DOI vorliegt, ist sie angegeben; bei Projektdokumentationen, Whitepapers und Beiträgen ohne vergebene DOI steht stattdessen die stabile Quellenangabe.

Erklärung zur Nutzung von KI-Werkzeugen

Diese Arbeit wurde unter Einsatz eines KI-Assistenzsystems (Claude, Anthropic) erstellt. Das System wurde für die sprachliche Ausarbeitung des Textes aus den Vorgaben des Autors, für Literaturrecherche, für die Formalisierung der Herleitungen in Anhang B sowie für Referenzcode, Simulation und Satz eingesetzt. Konzept, Architekturentscheidungen und Protokollparameter stammen vom Autor. Die Ergebnisse wurden vom Autor geprüft, jedoch nicht durchgängig unabhängig repliziert. Der Quellcode ist eine Referenzimplementierung ohne Produktionsreife und ohne externes Sicherheitsaudit; die Simulationen beruhen auf modellhaften Annahmen, nicht auf Betriebsdaten. Der Autor trägt die volle Verantwortung für den Inhalt.